

Sumário

I	PARTE 1: MODELAGEM DE DADOS (RA1)	3
1	INFORMAÇÕES GERAIS DO PROJETO	4
1.1	Título do Sistema	4
1.2	Integrantes da Equipe	4
2	DESCRIÇÃO DO MODELO DE NEGÓCIO	5
2.1	Itens do Modelo de Negócio	5
3	JUSTIFICATIVAS TÉCNICAS DA MODELAGEM	6
3.1	Relacionamentos	6
3.2	Chaves Primárias (PK)	6
3.3	Chaves Estrangeiras (FK)	6
3.4	Atributos e Domínios	6
3.5	Normalização	7
3.6	Especializações/Generalizações (Herança)	7
3.7	Decisões de Desempenho e Escalabilidade	7
3.8	Alternativas Consideradas	7
4	MODELOS E SCRIPTS	8
4.1	Modelo Conceitual – Diagrama Entidade-Relacionamento	8
4.2	Modelo Lógico e Físico - Script SQL de Criação	9
4.3	Evidências Adicionais	9
II	PARTE 2: IMPLEMENTAÇÃO E CONSULTAS (RA2)	10
5	IMPLEMENTAÇÃO E POPULAÇÃO DO BANCO	11
5.1	Script SQL de Inserção de Dados (População)	11

6	SCRIPTS SQL DE CONSULTAS OBRIGATÓRIAS	12
6.1	Consultas com JOIN (Mínimo 10)	12
6.2	Consultas com GROUP BY e HAVING (Mínimo 5)	12
6.3	Subqueries (Mínimo 5)	12
6.4	Criação de VIEWS (Mínimo 5)	13
6.5	UPDATEs e DELETEs Justificados	13
6.5.1	UPDATEs	13
6.5.2	DELETE	13
7	RELATÓRIO EXPLICATIVO DAS CONSULTAS PROPOSTAS PELO PROFESSOR	14
7.1	Ajustes e Melhorias na Modelagem (Revisão da Parte 1)	14
III	PARTE 3: CONTROLE DE ACESSO, TRANSAÇÕES E OTIMIZAÇÃO (RA3)	15
8	SEGURANÇA E CONTROLE DE ACESSO	16
8.1	Criação de Usuários e Perfis de Acesso (DCL)	16
8.2	Simulação de Revogação de Acesso	16
9	CONTROLE DE TRANSAÇÕES (COMMIT/ROLLBACK)	18
9.1	Simulação de Transação de Venda (Sucesso)	18
9.2	Simulação de Transação com Falha (Rollback)	18
10	OTIMIZAÇÃO DE DESEMPENHO E BOAS PRÁTICAS	20
10.1	Análise de Índices	20
10.2	Configuração de Backup e Log	20
	Declaração de Autoria	21

Parte I

Parte 1: Modelagem de Dados (RA1)

1 Informações Gerais do Projeto

1.1 Título do Sistema

Sistema de Compras Online (Lojas Online)

1.2 Integrantes da Equipe

- Allan Takeshi Morimoto
- Gustavo Vinícius de Claman de Andrade
- Heitor Alves dos Santos
- João Rafael Kuzmicz Andrade
- Thiago Muraro de Paula Gomes

2 Descrição do Modelo de Negócio

2.1 Itens do Modelo de Negócio

- **Identificação da Organização:** Nós da 4TAGH somos um grupo de pequeno porte que atua na área de banco de dados.
- **Objetivo Principal:** Nosso principal objetivo é projetar um serviço de compras online que permite ao usuário cadastrar uma conta, navegar por um catálogo de produtos organizados em categorias, adicionar itens ao carrinho e acompanhar seu status até a entrega no endereço cadastrado com o viés de facilitar o processo de compras e vendas entre o comprador e o vendedor, rastrear pacotes para a segurança de ambos os lados e emitir o acompanhamento da entrega durante todo o processo de envio.
- **Público-alvo e Usuários:** Os principais usuários do nosso projeto serão todos os interessados em publicar seus produtos ou em comprar os demais disponíveis (comprador e vendedor como já mencionado antes).
- **Processos Principais:** As principais entidades do nosso sistema são: Conta, Categoria, Carrinho, Loja, Produtos, Endereço, Estoque, Histórico, Pedidos e Pagamento. Além disso, os principais processos são o cadastro para garantir que os dados serão armazenados em uma conta, histórico para além de gerar um algoritmo voltado ao gosto do cliente também garantir o gerenciamento de compras entregues ou em envio, faturamento por meio da emissão de pagamentos realizados e verificados via entidade “pagamento” e estoque para garantir que não haverá como comprar um produto sem estoque.
- **Fluxo de Informações:** O sistema possibilita o gerenciamento de contas pessoais (histórico de compras, pedidos feitos ou entregues, Endereços cadastrados, formas de pagamento, etc) e contas comerciais (Loja cadastrada, produtos cadastrados, estoque disponível, categoria dos produtos, etc). Ademais, vendedores podem cadastrar e atualizar seus produtos, enquanto os clientes podem avaliá-los e visualizar opiniões de outros compradores. Dessa forma, controle de acesso por tipo de usuário (cliente ou vendedor), verificação de disponibilidade de estoque antes da venda e registro de todas as compras realizadas no histórico.
- **Necessidade do Banco de Dados:** O fluxo de dados é gerado a partir de cadastros feitos tanto pelos vendedores quanto pelos usuários. O nosso banco de dados é necessário uma vez que atualmente existem 3 diversas lojas virtuais que precisam de gerenciamento de informações. Dessa forma, tem o viés de fornecer segurança aos nossos usuários, garantir a privacidade de cada um e a disponibilidade dos produtos cadastrados.
- **Regras de Negócio:** As principais regras do negócio é uma unidade de CPF/CNPJ por consumidor/vendedor, não haverá um limite de crédito para os usuários e o cancelamento de algum produto poder ser realizado em um prazo útil determinado

2 Descrição do Modelo de Negócio

pelo vendedor. Ademais é OBRIGATÓRIO ter um cadastro no aplicativo para poder realizar um pedido ou uma venda.

- **Entidades e Relacionamentos-chave:**

Relacionamentos:

- Cliente possui uma conta, a conta possui obrigatoriamente a um cliente.
- O cliente tem obrigatoriamente um carrinho, o carrinho tem obrigatoriamente um cliente.
- O cliente pode ter pelo menos 1 produto no carrinho, um carrinho pode ter vários produtos.
- Um fornecedor pode ter vários produtos, um produto pode ter apenas um fornecedor.
- O cliente pode fazer vários pedidos, um pedido pode ser feito por um cliente.
- Um pedido pode ter vários produtos, um produto pode ter vários pedidos.
- A conta tem um carrinho e um histórico, o carrinho pode estar num histórico.
- O pedido tem um status, o status tem um pedido.
- O status de um pedido pode ser: preparo, pago, envio, entregue, perdido, cancelado, devolvido ou reembolsado. Dessa forma, perdido, devolvido e reembolsado podem coexistir. (no mínimo status 1, máximo 2).
- Todo pedido feito é adicionado ao histórico.
- Categoria tem vários produtos e o produto tem várias categorias.

- **Requisitos Não Funcionais Relevantes:**

Aspectos importantes:

- Perfis distintos de acesso (vendedor, usuário comum).
- Se o estoque do produto for igual a zero, então não será possível realizar a compra.
- Auditoria de devolução de produtos ou cancelamento de pedidos.
- Registro de logs com histórico de compras dos usuários para manipulação de algoritmo, favoritando as preferências do usuário.

2 Descrição do Modelo de Negócio

- **Relatórios e Consultas Esperadas:**

Funcionalidades esperadas:

- Analisar a disponibilidade do produto antes de vendê-lo.
- Gerar relatório do status de envio do produto.
- Atualizar as informações relevantes do produto antes da venda.
- Verificar se há uma conta vinculada a loja antes de realizar a compra do produto.
- Registrar pedidos em um carrinho virtual.
- Salvar as compras realizadas em um histórico.
- Controlar acesso por tipo de usuário (vendedor pode publicar seus produtos, mas não os comprar de volta).
- Gerenciar as opções de pagamento do cliente.

3 Justificativas Técnicas da Modelagem

3.1 Relacionamentos

Conta (1,1) — (1, N) Endereço

- Uma conta pode ter vários endereços (entrega / cobrança).
- Um endereço pertence a exatamente uma conta.
- Obrigatoriedade: Endereço.ID_Conta NOT NULL;

Conta pode existir sem endereço.

Conta (1,1) — (1,1) Carrinho

- Cada conta possui um carrinho ativo.
- Cada carrinho pertence a exatamente uma conta.
- Obrigatoriedade: Carrinho.ID_Conta NOT NULL;

Conta existe sem carrinho ativo até criar um.

Carrinho (1,1) — (0,1) Pedido

- Um carrinho pode gerar um pedido no momento do checkout.
- Um pedido tem origem em exatamente um carrinho (se desejar manter referência).
- Obrigatoriedade: Carrinho pode não ter pedido (se ainda não finalizado). Pedido pode referenciar o ID_Carrinho (opcional).

Carrinho (1,1) — (0, N) Produtos

- Um carrinho pode conter vários produtos; um produto pode estar em vários carrinhos.
- Implementação prática: manter tabela associativa (ex.: CarrinhoItem) com ID_Carrinho, ID_Produto, quantidade, preco_snapshot.

Conta (1,1) — (0, N) Pedidos

- Uma conta pode fazer vários pedidos.
- Cada pedido é feito por exatamente uma conta.
- Obrigatoriedade: Pedido.ID_Conta NOT NULL.

3 Justificativas Técnicas da Modelagem

Pedido (1,1) — (N,1) Pagamento

- Um pedido pode ter um ou mais registros de pagamento (ex.: parcelas, reembolso parcial).
- Cada pagamento está vinculado a um único pedido.
- Obrigatoriedade: Pagamento.ID_Pedido NOT NULL.

Endereço (1,N) — (1,N) Pedido

- Cada pedido só pode ter um endereço. 11
- Um endereço pode ser usado em vários pedidos.
- Obrigatoriedade: Pedido.ID_Endereco NOT NULL.

Pedido (1,N) — (1,N) Produtos

- Um pedido pode conter vários produtos; um produto pode fazer parte de vários pedidos.
- Implementação prática: usar tabela associativa (ex.: ItemPedido) com ID_Pedido, ID_Produto, quantidade, preco_unitario, subtotal.

Produtos (1,N) — (1,N) Categorias

- Um produto pode estar em várias categorias; uma categoria contém vários produtos.
- Implementação prática: tabela associativa (ex.: ProdutoCategoria) com ID_Produto, ID_Categoria.

Conta (1,1) — (0,N) HistóricoCompras

- Cada conta possui várias entradas no histórico de compras (uma por pedido).
- Cada entrada de HistóricoCompras refere-se a uma única conta e idealmente a um único pedido (FK ID_Pedido).
- Obrigatoriedade: Histórico.ID_Conta NOT NULL; Histórico.ID_Pedido NOT NULL (se for registro 5 por pedido).

3.2 Chaves Primárias (PK)

3 Justificativas Técnicas da Modelagem

● Conta:

PK ID_Conta (int, auto_increment)

- Critério de escolha: Artificial/Substituta (surrogate key).
- Motivo para não utilizar outra opção possível: O atributo email (chave candidata natural) seria maior e mutável. O ID_Conta garante um tamanho menor.
- Impacto na integridade e no desempenho:
Integridade Alta.
Desempenho Alto: O tipo int é o mais rápido para busca, junção (JOIN) e indexação.

● Carrinho:

PK ID_Carrinho (int, auto_increment)

- Critério de escolha: artificial (código de produto como SKU ou UUID), indicado por varchar.
- Motivo para não utilizar outra opção possível: Evitar uma PK composta (como ID_Conta + ID_Produto) e garantir uma chave simples e não composta para referência.
- Impacto na integridade e no desempenho:
Integridade alta.
Desempenho alto: int auto_increment é ideal para desempenho.

● Produto

PK ID_Produtos (varchar(45))

- Critério de escolha: Artificial (código de produto como SKU ou UUID), indicado por varchar.
- Motivo para não utilizar outra opção possível: O nome do produto não é único e pode mudar. O varchar é usado para um código padronizado ou significado externo.
- Impacto na integridade e no desempenho:
Integridade média.
Desempenho médio/baixo: Chaves varchar são mais lentas de indexar, comparar e referenciar que chaves int.

● Categoria

PK ID_Categorias (varchar(45))

- Critério de escolha: Artificial (código alfanumérico ou UUID).
- Motivo para não utilizar outra opção possível: O nome pode mudar. O ID dedicado evita que alterações no nome afetem chaves estrangeiras.

3 Justificativas Técnicas da Modelagem

- Impacto na integridade e no desempenho:
Integridade média.
Desempenho médio/baixo: varchar é mais lento.

- Loja:

PK CNPJ (char(14))

- Critério de escolha: Natural. É um identificador legal e único de domínio (mundo real).
- Motivo para não utilizar outra opção possível: O CNPJ é um identificador fortíssimo. Usá-lo como PK garante a unicidade de domínio e evita a duplicação de entidades do mundo real.
- Impacto na integridade e no desempenho:
Integridade alta.
Desempenho médio/baixo: char(14) é mais lento de comparar e usar em FKs do que int.

- Estoque:

PK ID_Estoque (varchar(45))

- Critério de escolha: Artificial.
- Motivo para não utilizar outra opção possível: Evitar uma chave composta (como CNPJ + Número_Estoque) e facilitar a referência por Produtos com um ID simples.
- Impacto na integridade e no desempenho:
Integridade média/alta.
Desempenho médio/baixo devido ao varchar.

- Pedidos:

PK ID_Pedidos (varchar(45))

- Critério de escolha: Artificial. Usado como número de pedido externamente referenciável.
- Motivo para não utilizar outra opção possível: Evitar uma chave composta (como ID_Conta + data_pedido + número_sequencial) e fornecer um ID único e simples para o usuário.
- Impacto na integridade e no desempenho:
Integridade alta.

3 Justificativas Técnicas da Modelagem

Desempenho médio/baixo: varchar é mais lento que int.

- Endereço:

PK ID_Endereço (int)

- Critério de escolha: Artificial/Substituta (Surrogate Key).
- Motivo para não utilizar outra opção possível: A chave natural (endereço completo) seria composta e longa (vários varchar), tornando-a ineficiente e complexa para referências.
- Impacto na integridade e no desempenho:
Integridade alta.
Desempenho Alto: O tipo int é ideal para indexação, junção e FKs.

- Pagamento:

PK ID_Pagamento (varchar(45))

- Critério de escolha: Artificial.
- Motivo para não utilizar outra opção possível: Separar o pagamento do pedido (1:1 ou 1:N) e usar um ID que pode ser gerado externamente ou para rastreabilidade de transação.
- Impacto na integridade e no desempenho:
Integridade alta.
Desempenho médio/baixo devido ao varchar.

- Histórico Compras:

PK ID_Historico (varchar(45))

- Critério de escolha: Artificial.
- Motivo para não utilizar outra opção possível: A descrição sugere um registro de histórico. Um ID artificial é usado para identificar o registro de histórico de forma única.
- Impacto na integridade e no desempenho:
Integridade alta.

3 Justificativas Técnicas da Modelagem

Desempenho médio/baixo devido ao varchar.

- Itens vendidos:

PK ID_Item (int, auto_increment)

- Critério de escolha: Artificial/Substituta.
- Motivo para não utilizar outra opção possível: A chave natural seria a composta (CNPJ, ID_Produtos, data_venda). Essa chave é excessivamente longa e complexa, o que a torna ineficiente para indexação e referências. O uso de um ID_Item simples e inteiro garante máxima eficiência.
- Integridade alta.
Desempenho altíssimo: O tipo int é o mais rápido para operações de inserção, busca e referência. É ideal para tabelas de transação que crescem muito rapidamente.

3.3 Chaves Estrangeiras (FK)

- Carrinho:

- FK: ID_Conta (Cada Conta possui um carrinho próprio)

- Motivo de inclusão: Cada carrinho deve estar vinculado a uma conta específica, garantindo que apenas um usuário seja responsável por ele.
- Relação com a PK referenciada: ID_Conta é a chave primária da entidade *Conta*.
- Efeito sobre integridade referencial: Impede que um carrinho exista sem estar associado a uma conta válida. Caso uma conta seja removida, o sistema deve definir regra (ex.: cascade delete ou impedir a exclusão).

- Produtos:

- FK: ID da categoria (Cada produto pertence a uma categoria específica)

- FK: ID_Estoque (cada produto possui um estoque próprio)

- FK: ID_Categoria

3 Justificativas Técnicas da Modelagem

- Motivo: Cada produto pertence a uma única categoria.
- Relação: Aponta para a PK de Categoria.
- Integridade: Garante que produtos só existam dentro de categorias cadastradas.
- FK: ID_Estoque
 - Motivo: Cada produto possui um estoque individual.
 - Relação: Aponta para a PK da entidade Estoque.
 - Integridade: Assegura que um produto só seja cadastrado se existir o registro de estoque correspondente.
- Loja:
 - FK: ID_Endereço (Cada loja possui um endereço físico próprio)
 - Motivo: Cada loja precisa de um endereço físico único.
 - Relação: Aponta para a PK de Endereço.
 - Integridade: Impede lojas com endereço inexistente; evita duplicidade indevida de endereços.
- Estoque:
 - FK: CNPJ (Cada estoque pertence á uma loja)
 - Motivo: Todo estoque pertence a uma loja específica.
 - Relação: CNPJ é a PK da entidade Loja.
 - Integridade: Evita criar estoque sem loja vinculada.
- Pedidos:
 - FK: ID da conta (Um pedido esta OBRIGATORIAMENTE vinculado a uma conta)
 - Motivo: Todo pedido deve pertencer a uma conta (usuário).
 - Relação: Aponta para a PK de Conta.
 - Integridade: Garante que nenhum pedido exista sem usuário responsável.

3 Justificativas Técnicas da Modelagem

- **Endereço:**

- FK: ID_Conta (cada endereço pertence a uma conta única)
 - Motivo: Um endereço pertence a uma conta única.
 - Relação: Aponta para a PK de Conta.
 - Integridade: Impede o cadastro de um endereço sem um usuário associado.

- **Pagamento:**

- FK:ID_pedidos (um pagamento pertence a um pedido)
 - Motivo: O pagamento está sempre vinculado a um pedido específico.
 - Relação: Aponta para a PK de Pedidos.
 - Integridade: Garante correspondência entre pedido e pagamento, evitando pagamentos soltos.

- **Histórico Compras:**

- FK: ID_Conta (um histórico pertence á uma conta única)
 - Motivo: Cada histórico pertence a um único usuário.
 - Relação: Aponta para a PK de Conta.
 - Integridade: Só permite histórico se houver conta válida vinculada.

3.4 Atributos e Domínios

IDs (PK e FK):

Tipo: INT ou BIGINT

Motivo: Inteiros são eficientes para buscas e indexação.

CNPJ:

Tipo: CHAR(14)

3 Justificativas Técnicas da Modelagem

Motivo: Formato fixo; evita inconsistências de tamanho.

Nomes (conta, produto, loja, categoria):

Tipo: VARCHAR(100 a 255)

Motivo: Permite flexibilidade em nomes com tamanhos variados.

Descrição de produto:

Tipo: TEXT

Motivo: Texto longo e variável.

Preço:

Tipo: DECIMAL(10,2)

Motivo: Precisão necessária para valores monetários, evitando erros de ponto flutuante.

Quantidade em estoque:

Tipo: INT

Motivo: Representa valores inteiros; adequado para contagem de itens.

Datas (pedido, pagamento, histórico):

Tipo: DATE ou DATETIME

Motivo: Necessidade de registro temporal confiável.

NOT NULL

Usado para atributos obrigatórios.

Justificativa: Garantir que informações críticas existam sempre.

UNIQUE

3 Justificativas Técnicas da Modelagem

Usado quando não pode haver duplicação.

Justificativa: Evitar inconsistências e duplicidade em dados sensíveis.

CHECK

Usado para impor regras específicas.

Justificativa: Garante validade lógica e consistência interna dos dados.

3.5 Normalização

Aplicação 1FN:

- Atributos sem campos compostos ou multivalorados.
- Cada coluna possui um único valor por registro.
- Cada tabela possui chave primária definida

Aplicação 2FN:

- A 2FN exige que não haja dependência funcional parcial da PK em relação aos atributos não chave.
- Como todas as tabelas usam chaves primárias simples, não há dependências parciais.

Aplicação 3FN:

- O endereço foi separado da conta.
- O estoque foi separado dos produtos.
- Categoria é uma entidade própria, não armazenada diretamente dentro de *produtos*.
- Pagamento foi separado de pedidos, evitando misturar informações financeiras com o pedido em si.

3.6 Especializações/Generalizações (Herança)

Em um modelo avançado, poderíamos dividir em:

- **Conta (superclasse)**
- **Cliente (subclasse)**
- **Vendedor (subclasse)**

Motivo para possível uso de herança: Separar atributos e funções específicas de vendedores (como CNPJ, número de registro, comissão etc.). Diferenciar permissões e papéis no sistema. Evitar muitos atributos opcionais ou nulos na mesma tabela.

A herança seria aplicada se:

- Existirem atributos exclusivos para vendedores.
- Existirem funcionalidades distintas (ex.: dashboards, relatórios, cadastro de produtos).
- O sistema crescer ao ponto de separar responsabilidades lógicas.

3.7 Decisões de Desempenho e Escalabilidade

- **Índice no email da conta (UNIQUE)**

Melhora login e autenticação.

- **Índice em ID_Conta nas tabelas:**

melhora JOINS frequentes.

- **Índice em ID_Categoria dos produtos**

melhora filtro por categoria na loja.

- **Índice em nome do produto**

melhora pesquisas no catálogo.

3.8 Alternativas Consideradas

1. Unir estoque dentro de produtos

Descartado porque:

- Estoque pode ser separado em múltiplos locais no futuro.
- Evita inconsistências ao atualizar quantidade.

2. Armazenar endereço embutido na conta

Descartado porque:

- Uma conta pode ter mais de um endereço.
- Reduz duplicação e facilita atualização.

4.1 Modelo Conceitual – Diagrama Entidade-Relacionamento

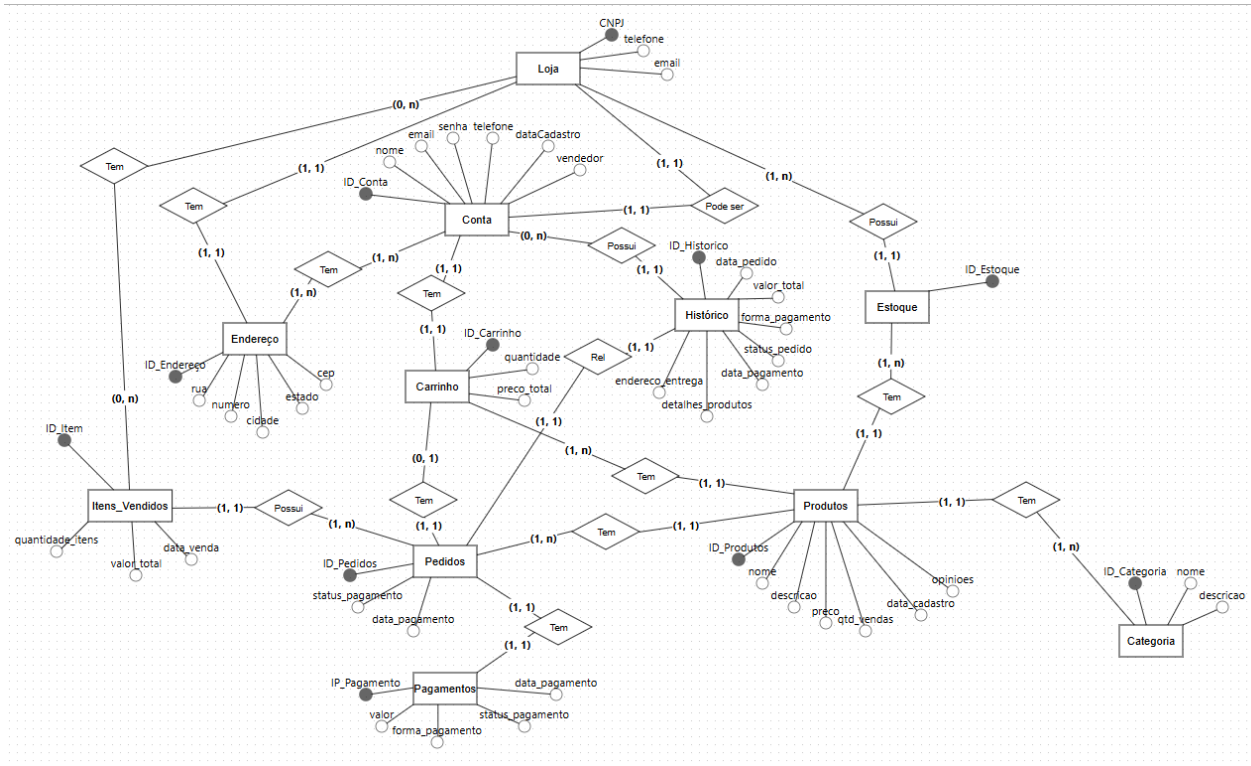
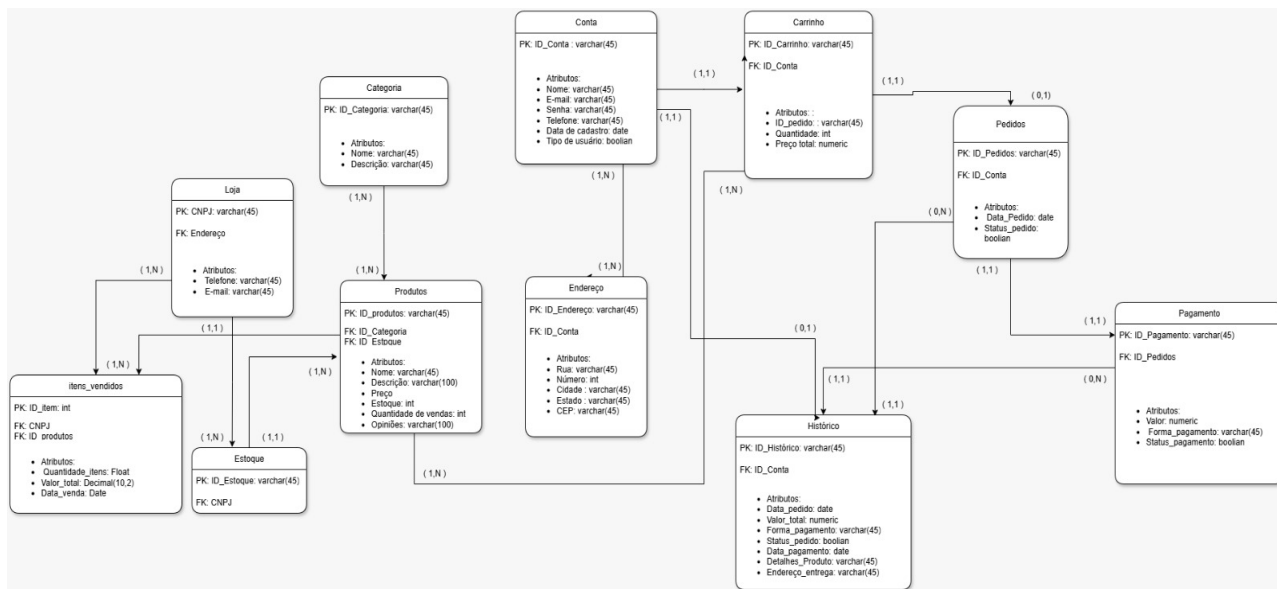


Figura 1 – Diagrama Entidade-Relacionamento (Modelo Conceitual)

<https://app.brmodeloweb.com/#!/publicview/69210a6039eddf537c9b004d>

5.1 Modelo Lógico - Script SQL de Criação

Inclua o modelo lógico (tabelas, atributos, tipos de dados, restrições) e o script SQL de criação das tabelas.



6.1 Modelo Físico - Script SQL de Criação

```
CREATE TABLE conta (
ID_Conta int NOT NULL PRIMARY KEY,
nome varchar(255) NOT NULL,
email varchar(255),
senha varchar(255) NOT NULL,
telefone varchar(20),
dataCadastro DATETIME,
vendedor bool
);
```

```
CREATE TABLE endereco (
ID_Endereco INT NOT NULL PRIMARY KEY,
ID_Conta INT NOT NULL,
rua VARCHAR(100) NOT NULL,
numero INT NOT NULL,
cidade VARCHAR(100) NOT NULL,
estado CHAR(2) NOT NULL,
cep CHAR(8) NOT NULL,
CONSTRAINT FK_Endereco_Conta FOREIGN KEY (ID_Conta) REFERENCES conta(ID_Conta)
);
```

```
CREATE TABLE categoria (  
ID_Categoria varchar(45) NOT NULL PRIMARY KEY,  
nome varchar(45) NOT NULL,  
descricao varchar(150)  
);
```

```
CREATE TABLE carrinho (  
ID_Carrinho INT NOT NULL PRIMARY KEY,  
ID_Conta INT NOT NULL,  
ID_Pedido varchar(45) NOT NULL,  
quantidade INT NOT NULL,  
preco_total DECIMAL(10,2) NOT NULL,  
CONSTRAINT FK_Carrinho_Conta FOREIGN KEY (ID_Conta) REFERENCES conta(ID_Conta)  
CONSTRAINT FK_Carrinho_Pedido FOREIGN KEY (ID_Pedido) REFERENCES pedidos(ID_Pedido)  
);
```

```
CREATE TABLE loja (  
CNPJ char(14) NOT NULL PRIMARY KEY,  
ID_Endereco INT NOT NULL,  
telefone varchar(12) NOT NULL,  
email varchar(255),  
CONSTRAINT FK_Loja_Endereco FOREIGN KEY (ID_Endereco) REFERENCES  
endereco(ID_Endereco)  
);
```

```
CREATE TABLE estoque (  
ID_Estoque varchar(45) NOT NULL PRIMARY KEY,  
CNPJ char(14) NOT NULL,  
CONSTRAINT FK_Estoque_CNPJ FOREIGN KEY (CNPJ) REFERENCES loja(CNPJ)  
);
```

```
CREATE TABLE produtos (  
ID_Produtos VARCHAR(45) NOT NULL PRIMARY KEY,  
ID_Categoria VARCHAR(45) NOT NULL,  
ID_Estoque varchar(45) NOT NULL,  
nome VARCHAR(45) NOT NULL,  
descricao VARCHAR(150),  
preco decimal(10,2) NOT NULL,  
estoque INT NOT NULL,  
qtd_vendas INT DEFAULT 0,  
data_cadastro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
opinioes VARCHAR(150),  
CONSTRAINT FK_Produtos_Categoria FOREIGN KEY (ID_Categoria) REFERENCES  
categoria(ID_Categoria),  
CONSTRAINT FK_Produtos_Estoque FOREIGN KEY (ID_Estoque) REFERENCES estoque(ID_Estoque)  
);
```

```
CREATE TABLE pedidos (  
  ID_Pedidos VARCHAR(45) NOT NULL PRIMARY KEY,  
  ID_Conta INT NOT NULL,  
  data_pedido DATE NOT NULL,  
  status_pedido bool NOT NULL,  
  CONSTRAINT FK_Pedidos_Conta FOREIGN KEY (ID_Conta) REFERENCES conta(ID_Conta)  
);
```

```
CREATE TABLE pagamento (  
  ID_Pagamento VARCHAR(45) NOT NULL PRIMARY KEY,  
  ID_Pedidos VARCHAR(45) NOT NULL,  
  valor decimal(10,2) NOT NULL,  
  forma_pagamento VARCHAR(45) NOT NULL,  
  status_pagamento bool NOT NULL,  
  data_pagamento DATE NOT NULL,  
  CONSTRAINT FK_Pagamento_Pedidos FOREIGN KEY (ID_Pedidos) REFERENCES  
  pedidos(ID_Pedidos)  
);
```

```
CREATE TABLE historico_Compras (  
  ID_Historico VARCHAR(45) NOT NULL PRIMARY KEY,  
  ID_Conta INT NOT NULL,  
  data_pedido DATE NOT NULL,  
  valor_total DECIMAL(10,2) NOT NULL,  
  forma_pagamento VARCHAR(45) NOT NULL,  
  status_pedido bool NOT NULL,  
  data_pagamento DATE,  
  detalhes_produtos VARCHAR(150),  
  endereco_entrega VARCHAR(45),  
  CONSTRAINT FK_Historico_Conta FOREIGN KEY (ID_Conta) REFERENCES conta(ID_Conta)  
);
```

```
CREATE TABLE itens_vendidos (  
  ID_Item INT AUTO_INCREMENT PRIMARY KEY,  
  CNPJ CHAR(14) NOT NULL,  
  ID_Produtos VARCHAR(45) NOT NULL,  
  quantidade_itens FLOAT NOT NULL,  
  valor_total DECIMAL(10,2) NOT NULL,  
  data_venda DATE NOT NULL,  
  CONSTRAINT FK_Loja_CNPJ FOREIGN KEY (CNPJ) REFERENCES loja(CNPJ),  
  CONSTRAINT FK_Produto_ID FOREIGN KEY (ID_Produtos) REFERENCES produtos(ID_Produtos)  
);
```

7.1 Evidências Adicionais

Navigator

SCHEMAS

Filter objects

atividade

- Tables
 - categoria
 - carrinho
 - conta
 - endereço
 - loja
- Views
- Stored Procedures
- Functions

sakila

sys

world

Administration Schemas

Information

No object selected

Object Info Session

Filter objects

atividade

- Tables
 - categoria
 - carrinho
 - conta
 - endereço
 - estoque
 - loja
 - pagamento
 - pedidos
 - produtos
- Views
- Stored Procedures
- Functions

sakila

sys

world

Administration Schemas

Query 1 SQL File 1* SQL File 2* SQL File 3* SQL File 4*

Limit to 1000 rows

```

1 CREATE TABLE categoria (
2     ID_Categoria varchar(45) NOT NULL PRIMARY KEY,
3     nome varchar(45) NOT NULL,
4     descricao varchar(150)
5 );

```

Output

Action Output

#	Time	Action	Message
6	22:43:22	create table loja(CNPJ char(14) NOT NULL PRIMARY KEY, ID_Endereco INT NOT NULL, telefone varchar(12) NOT ...	0 row(s) affecte
7	22:52:03	DROP TABLE 'atividade'. 'caminho'	0 row(s) affecte
8	22:52:24	CREATE TABLE caminho (ID_Caminho INT NOT NULL PRIMARY KEY, ID_Conta INT NOT NULL, ID_Pedid...	0 row(s) affecte
9	22:55:16	CREATE TABLE categoria (ID_Categoria varchar(45) NOT NULL PRIMARY KEY, nome varchar(45) NOT NULL...	Error Code: 105
10	22:55:28	DROP TABLE 'atividade'. 'categoria'	0 row(s) affecte
11	22:55:29	CREATE TABLE categoria (ID_Categoria varchar(45) NOT NULL PRIMARY KEY, nome varchar(45) NOT NULL...	0 row(s) affecte

Limit to 1000 rows

```

1 CREATE TABLE pagamento (
2     ID_Pagamento VARCHAR(45) NOT NULL PRIMARY KEY,
3     ID_Pedidos VARCHAR(45) NOT NULL,
4     valor decimal(10,2) NOT NULL,
5     forma_pagamento VARCHAR(45) NOT NULL,
6     status_pagamento bool NOT NULL, -- 0 = pendente, 1 = concluído
7     data_pagamento DATE NOT NULL,
8     CONSTRAINT FK_Pagamento_Pedidos FOREIGN KEY (ID_Pedidos)
9     REFERENCES pedidos(ID_Pedidos)
10 );
11

```


Schema: atividade

```

1 CREATE TABLE carrinho (
2     ID_Carrinho INT NOT NULL PRIMARY KEY,
3     ID_Conta INT NOT NULL,
4     ID_Pedido varchar(45) NOT NULL,
5     quantidade INT NOT NULL,
6     preco_total DECIMAL(10,2) NOT NULL,
7     CONSTRAINT FK_Carrinho_Conta FOREIGN KEY (ID_Conta)
8     REFERENCES conta(ID_Conta)
9 );

```

#	Time	Action	Message
3	22:35:31	CREATE TABLE endereco (ID_Endereco INT NOT NULL PRIMARY KEY, ID_Conta INT NOT NULL, rua VA...	0 row(s) affected
4	22:35:49	CREATE TABLE categoria (ID_Categoria int NOT NULL PRIMARY KEY, nome varchar(255) NOT NULL, des...	0 row(s) affected
5	22:38:04	CREATE TABLE carrinho (ID_Carrinho INT NOT NULL PRIMARY KEY, ID_Conta INT NOT NULL, ID_Pedid...	0 row(s) affected
6	22:43:22	create table loja(CNPJ char(14) NOT NULL PRIMARY KEY, ID_Endereco INT NOT NULL, telefone varchar(12) NOT ...	0 row(s) affected
7	22:52:03	DROP TABLE 'atividade'.carrinho	0 row(s) affected
8	22:52:24	CREATE TABLE carrinho (ID_Carrinho INT NOT NULL PRIMARY KEY, ID_Conta INT NOT NULL, ID_Pedid...	0 row(s) affected

Navigator

SCHEMAS

Filter objects

- atividade
 - Tables
 - categoria
 - carrinho
 - conta
 - endereco
 - loja
 - Views
 - Stored Procedures
 - Functions
- sakila
- sys
- world

Administration Schemas Information

No object selected

Object Info Session

Query 1 SQL File 1* SQL File 2* SQL File 3* SQL File 4*

Limit to 1000 rows

```
1 CREATE TABLE categoria (  
2     ID_Categoria varchar(45) NOT NULL PRIMARY KEY,  
3     nome varchar(45) NOT NULL,  
4     descricao varchar(150)  
5 );
```

Output

Action Output

#	Time	Action	Message
6	22:43:22	create table loja(CNPJ char(14) NOT NULL PRIMARY KEY, ID_Endereco INT NOT NULL, telefone varchar(12) NOT NULL)	0 row(s) affected
7	22:52:03	DROP TABLE 'atividade'. 'carrinho'	0 row(s) affected
8	22:52:24	CREATE TABLE carrinho (ID_Carrinho INT NOT NULL PRIMARY KEY, ID_Conta INT NOT NULL, ID_Pedido INT NOT NULL)	0 row(s) affected
9	22:55:16	CREATE TABLE categoria (ID_Categoria varchar(45) NOT NULL PRIMARY KEY, nome varchar(45) NOT NULL, descricao varchar(150))	Error Code: 105
10	22:55:28	DROP TABLE 'atividade'. 'categoria'	0 row(s) affected
11	22:55:29	CREATE TABLE categoria (ID_Categoria varchar(45) NOT NULL PRIMARY KEY, nome varchar(45) NOT NULL, descricao varchar(150))	0 row(s) affected

Parte II

Parte 2: Implementação e Consultas (RA2)

8 Implementação e População do Banco

8.1 Script SQL de Inserção de Dados (População)

INSERT INTO conta (ID_Conta, nome, email, senha, telefone, dataCadastro, vendedor) VALUES

```
(1, 'Carlos Silva', 'carlos.silva@example.com', 'senha123', '11987654321', '2025-10-07 08:30:00', true),
(2, 'Ana Pereira', 'ana.pereira@example.com', 'anaSenha@2025', '11987654322', '2025-09-20 14:22:00', false),
(3, 'João Costa', 'joao.costa@example.com', 'joao2025!', '11987654323', '2025-09-15 09:15:00', true),
(4, 'Mariana Rocha', 'mariana.rocha@example.com', 'senhaMariana1', '11987654324', '2025-09-01 10:00:00', false),
(5, 'Rafael Souza', 'rafael.souza@example.com', 'rafael2025', '11987654325', '2025-08-20 16:45:00', true),
(6, 'Luana Almeida', 'luana.almeida@example.com', 'luanaSenha2025', '11987654326', '2025-07-30 11:30:00', false),
(7, 'Pedro Martins', 'pedro.martins@example.com', 'pedro2025!', '11987654327', '2025-06-22 18:10:00', true),
(8, 'Patrícia Oliveira', 'patricia.oliveira@example.com', 'patricia123', '11987654328', '2025-06-05 13:25:00', false),
(9, 'Lucas Pereira', 'lucas.pereira@example.com', 'lucasSenha2025', '11987654329', '2025-05-15 12:05:00', true),
(10, 'Sofia Costa', 'sofia.costa@example.com', 'sofia@2025!', '11987654330', '2025-04-30 17:50:00', false),
(11, 'Carlos Lima', 'carlos.lima@example.com', 'carlos2025@', '11987654331', '2025-03-28 15:15:00', true),
(12, 'Bárbara Ferreira', 'barbara.ferreira@example.com', 'barbara123', '11987654332', '2025-03-12 09:30:00', false),
(13, 'Thiago Gomes', 'thiago.gomes@example.com', 'senhaThiago@2025', '11987654333', '2025-02-05 14:45:00', true),
(14, 'Isabela Nunes', 'isabela.nunes@example.com', 'isabelaSenha2025', '11987654334', '2025-01-21 11:00:00', false),
(15, 'Fernando Dias', 'fernando.dias@example.com', 'senha2025#', '11987654335', '2025-01-10 08:40:00', true),
(16, 'Fernanda Silva', 'fernanda.silva@example.com', 'fernanda123!', '11987654336', '2024-12-15 10:50:00', false),
(17, 'Ricardo Souza', 'ricardo.souza@example.com', 'ricardo2025$', '11987654337', '2024-11-30 13:05:00', true),
(18, 'Larissa Lima', 'larissa.lima@example.com', 'larissaSenha2025', '11987654338', '2024-10-15 16:00:00', false),
(19, 'Gustavo Costa', 'gustavo.costa@example.com', 'gustavo@2025', '11987654339', '2024-09-05 17:30:00', true),
(20, 'Raquel Almeida', 'raquel.almeida@example.com', 'raquel123!', '11987654340', '2024-08-12 19:00:00', false);
```

INSERT INTO endereco (ID_Endereco, ID_Conta, rua, numero, cidade, estado, cep) VALUES

```
(1, 1, 'Rua das Palmeiras', 123, 'São Paulo', 'SP', '01023000'),
(2, 2, 'Avenida Brasil', 456, 'Rio de Janeiro', 'RJ', '20040010'),
(3, 3, 'Rua Sete de Setembro', 789, 'Belo Horizonte', 'MG', '30100001'),
```

(4, 4, 'Rua da Paz', 101, 'Curitiba', 'PR', '80010000'),
 (5, 5, 'Avenida dos Andradas', 202, 'Porto Alegre', 'RS', '90035000'),
 (6, 6, 'Rua Joaquim Nabuco', 303, 'Fortaleza', 'CE', '60030000'),
 (7, 7, 'Rua XV de Novembro', 404, 'Salvador', 'BA', '40060100'),
 (8, 8, 'Praça da Liberdade', 505, 'Brasília', 'DF', '70000000'),
 (9, 9, 'Rua dos Três Irmãos', 606, 'Natal', 'RN', '59020000'),
 (10, 10, 'Rua Dr. José de Alencar', 707, 'Recife', 'PE', '50015000'),
 (11, 11, 'Avenida dos Oceanos', 808, 'São Luís', 'MA', '65000000'),
 (12, 12, 'Rua do Sol', 909, 'Maceió', 'AL', '57000000'),
 (13, 13, 'Avenida da Paz', 1010, 'Aracaju', 'SE', '49000000'),
 (14, 14, 'Rua da Alegria', 1111, 'Cuiabá', 'MT', '78000000'),
 (15, 15, 'Rua da Esperança', 1212, 'Campo Grande', 'MS', '79000000'),
 (16, 16, 'Avenida dos Eucaliptos', 1313, 'Goiânia', 'GO', '74000000'),
 (17, 17, 'Rua São João', 1414, 'Manaus', 'AM', '69000000'),
 (18, 18, 'Rua das Acácias', 1515, 'Belém', 'PA', '66000000'),
 (19, 19, 'Avenida Amazonas', 1616, 'Vitória', 'ES', '29000000'),
 (20, 20, 'Rua do Comércio', 1717, 'Macapá', 'AP', '68900000'),
 (21, 17, 'Rua São Joaquim', 4114, 'Manaus', 'AM', '69900000'),
 (22, 18, 'Rua das Araras', 5115, 'Belém', 'PA', '66700000'),
 (23, 19, 'Avenida Etna', 6116, 'Vitória', 'ES', '29100000'),
 (24, 20, 'Rua do Churrasco', 7117, 'Macapá', 'AP', '68930000');

INSERT INTO categoria (ID_Categoria, nome, descricao) VALUES

('CAT001', 'Eletrônicos', 'Dispositivos eletrônicos e gadgets'),
 ('CAT002', 'Roupas', 'Vestuário masculino, feminino e infantil'),
 ('CAT003', 'Alimentos', 'Produtos alimentícios diversos'),
 ('CAT004', 'Livros', 'Livros de todos os gêneros e tipos'),
 ('CAT005', 'Móveis', 'Móveis para casa e escritório'),
 ('CAT006', 'Beleza', 'Produtos de beleza e cuidados pessoais'),
 ('CAT007', 'Esportes', 'Equipamentos e acessórios esportivos'),
 ('CAT008', 'Brinquedos', 'Brinquedos para crianças de várias idades'),
 ('CAT009', 'Automóveis', 'Peças e acessórios para veículos'),
 ('CAT010', 'Informática', 'Hardware, software e acessórios de informática'),
 ('CAT011', 'Jardinagem', 'Ferramentas e produtos para jardinagem'),
 ('CAT012', 'Casa e Decoração', 'Itens para decoração e utilidades domésticas'),
 ('CAT013', 'Saúde', 'Produtos relacionados à saúde e bem-estar'),
 ('CAT014', 'Papeleria', 'Materiais de escritório e papeleria'),
 ('CAT015', 'Telefonia', 'Celulares, smartphones e acessórios'),
 ('CAT016', 'Calçados', 'Sapatos, tênis e sandálias'),
 ('CAT017', 'Instrumentos Musicais', 'Instrumentos e acessórios musicais'),
 ('CAT018', 'Viagem', 'Artigos e acessórios para viagens'),
 ('CAT019', 'Pets', 'Produtos para cuidados e alimentação de animais'),
 ('CAT020', 'Ferramentas', 'Ferramentas manuais e elétricas para construção');

INSERT INTO carrinho (ID_Carrinho, ID_Conta, ID_Pedido, quantidade, preco_total) VALUES

(1, 1, 'PED001', 2, 150.00),
 (2, 2, 'PED002', 1, 75.50),
 (3, 3, 'PED003', 4, 320.00),
 (4, 4, 'PED004', 3, 210.00),
 (5, 5, 'PED005', 1, 100.00),

```
(6, 6, 'PED006', 5, 450.00),
(7, 7, 'PED007', 2, 120.00),
(8, 8, 'PED008', 3, 180.00),
(9, 9, 'PED009', 1, 90.00),
(10, 10, 'PED010', 6, 540.00),
(11, 11, 'PED011', 2, 160.00),
(12, 12, 'PED012', 3, 240.00),
(13, 13, 'PED013', 1, 70.00),
(14, 14, 'PED014', 4, 280.00),
(15, 15, 'PED015', 2, 140.00),
(16, 16, 'PED016', 3, 210.00),
(17, 17, 'PED017', 5, 375.00),
(18, 18, 'PED018', 1, 75.00),
(19, 19, 'PED019', 4, 300.00),
(20, 20, 'PED020', 2, 160.00);
```

INSERT INTO loja (CNPJ, ID_Endereco, telefone, email) VALUES

```
('12345678000101', 1, '11999990001', 'contato1@loja1.com'),
('22345678000102', 2, '11999990002', 'contato2@loja2.com'),
('32345678000103', 3, '11999990003', 'contato3@loja3.com'),
('42345678000104', 4, '11999990004', 'contato4@loja4.com'),
('52345678000105', 5, '11999990005', 'contato5@loja5.com'),
('62345678000106', 6, '11999990006', 'contato6@loja6.com'),
('72345678000107', 7, '11999990007', 'contato7@loja7.com'),
('82345678000108', 8, '11999990008', 'contato8@loja8.com'),
('92345678000109', 9, '11999990009', 'contato9@loja9.com'),
('10345678000110', 10, '11999990010', 'contato10@loja10.com'),
('11345678000111', 11, '11999990011', 'contato11@loja11.com'),
('12345678000112', 12, '11999990012', 'contato12@loja12.com'),
('13345678000113', 13, '11999990013', 'contato13@loja13.com'),
('14345678000114', 14, '11999990014', 'contato14@loja14.com'),
('15345678000115', 15, '11999990015', 'contato15@loja15.com'),
('16345678000116', 16, '11999990016', 'contato16@loja16.com'),
('17345678000117', 17, '11999990017', 'contato17@loja17.com'),
('18345678000118', 18, '11999990018', 'contato18@loja18.com'),
('19345678000119', 19, '11999990019', 'contato19@loja19.com'),
('20345678000120', 20, '11999990020', 'contato20@loja20.com');
```

INSERT INTO estoque (ID_Estoque, CNPJ) VALUES

```
('EST001', '12345678000101'),
('EST002', '22345678000102'),
('EST003', '32345678000103'),
('EST004', '42345678000104'),
('EST005', '52345678000105'),
('EST006', '62345678000106'),
('EST007', '72345678000107'),
('EST008', '82345678000108'),
('EST009', '92345678000109'),
('EST010', '10345678000110'),
('EST011', '11345678000111'),
```

```
( 'EST012', '12345678000112'),
( 'EST013', '13345678000113'),
( 'EST014', '14345678000114'),
( 'EST015', '15345678000115'),
( 'EST016', '16345678000116'),
( 'EST017', '17345678000117'),
( 'EST018', '18345678000118'),
( 'EST019', '19345678000119'),
( 'EST020', '20345678000120');
```

```
INSERT INTO produtos (ID_Produtos, ID_Categoria, ID_Estoque, nome, descricao, preco, estoque,
qtd_vendas, opinioes, data_cadastro) VALUES
```

```
( 'PROD001', 'CAT001', 'EST001', 'Smartphone A1', 'Smartphone com tela 6.1", câmera 12MP', 1500.00,
100, 0, 'Muito bom, recomendo!', DATE_SUB(CURDATE(), INTERVAL 245 DAY)),
( 'PROD002', 'CAT002', 'EST002', 'Camiseta Básica', 'Camiseta 100% algodão, tamanho M', 35.00, 200,
500, 'Confortável e durável', DATE_SUB(CURDATE(), INTERVAL 892 DAY)),
( 'PROD003', 'CAT003', 'EST003', 'Arroz Integral 1kg', 'Pacote de arroz integral, saudável', 10.00, 2, 1500,
'Ótima qualidade', DATE_SUB(CURDATE(), INTERVAL 901 DAY)),
( 'PROD004', 'CAT004', 'EST004', 'Livro SQL Básico', 'Aprenda SQL do básico ao avançado', 60.00, 80,
100, 'Livro excelente para iniciantes', DATE_SUB(CURDATE(), INTERVAL 22 DAY)),
( 'PROD005', 'CAT005', 'EST005', 'Mesa de Escritório', 'Mesa de madeira com estrutura metálica', 350.00,
7, 50, 'Resistente e bonita', DATE_SUB(CURDATE(), INTERVAL 56 DAY)),
( 'PROD006', 'CAT006', 'EST006', 'Creme Hidratante', 'Creme para pele seca com vitamina E', 25.00, 120,
220, 'Hidratação prolongada', DATE_SUB(CURDATE(), INTERVAL 67 DAY)),
( 'PROD007', 'CAT007', 'EST007', 'Bola de Futebol', 'Bola oficial tamanho 5', 90.00, 3, 180, 'Boa
qualidade', DATE_SUB(CURDATE(), INTERVAL 156 DAY)),
( 'PROD008', 'CAT008', 'EST008', 'Quebra-cabeça 1000 peças', 'Imagem de paisagem natural', 80.00, 70,
0, 'Desafiador e divertido', DATE_SUB(CURDATE(), INTERVAL 667 DAY)),
( 'PROD009', 'CAT009', 'EST009', 'Óleo para Motor 5W-30', 'Óleo sintético para motores', 120.00, 5, 90,
'Alta performance', DATE_SUB(CURDATE(), INTERVAL 234 DAY)),
( 'PROD010', 'CAT010', 'EST010', 'Teclado Mecânico', 'Teclado com switches azuis, retroiluminado',
450.00, 8, 75, 'Excelente resposta', DATE_SUB(CURDATE(), INTERVAL 289 DAY)),
( 'PROD011', 'CAT001', 'EST001', 'Fone Bluetooth', 'Fone sem fio com cancelamento de ruído', 300.00, 70,
110, 'Som claro e bateria duradoura', DATE_SUB(CURDATE(), INTERVAL 789 DAY)),
( 'PROD012', 'CAT002', 'EST002', 'Calça Jeans', 'Calça jeans masculina slim fit', 120.00, 90, 130,
'Confortável e estilosa', DATE_SUB(CURDATE(), INTERVAL 512 DAY)),
( 'PROD013', 'CAT003', 'EST003', 'Café Torrado 500g', 'Café em grãos torrado', 18.00, 300, 800, 'Sabor
intenso', DATE_SUB(CURDATE(), INTERVAL 923 DAY)),
( 'PROD014', 'CAT004', 'EST004', 'Guia de Viagem Europa', 'Dicas e roteiros turísticos', 60.00, 100, 35,
'Informações completas', DATE_SUB(CURDATE(), INTERVAL 1 DAY)),
( 'PROD015', 'CAT005', 'EST005', 'Cadeira Gamer', 'Ergonômica com ajuste de altura', 850.00, 25, 0,
'Muito confortável', DATE_SUB(CURDATE(), INTERVAL 89 DAY)),
( 'PROD016', 'CAT006', 'EST006', 'Shampoo Anticaspa', 'Fórmula suave para couro cabeludo', 22.00, 110,
210, 'Eficaz e cheiroso', DATE_SUB(CURDATE(), INTERVAL 5 DAY)),
( 'PROD017', 'CAT007', 'EST007', 'Raquete de Tênis', 'Leve e resistente para intermediários', 380.00, 4,
40, 'Boa pegada', DATE_SUB(CURDATE(), INTERVAL 3 DAY)),
( 'PROD018', 'CAT008', 'EST008', 'Boneca de Pano', 'Artesanal e segura para crianças', 55.00, 80, 75,
'Muito fofa', DATE_SUB(CURDATE(), INTERVAL 19 DAY)),
( 'PROD019', 'CAT009', 'EST009', 'Bateria Automotiva 60Ah', 'Alta durabilidade e desempenho', 600.00, 6,
50, 'Funcionou perfeitamente', DATE_SUB(CURDATE(), INTERVAL 25 DAY)),
( 'PROD020', 'CAT010', 'EST010', 'Monitor 24"', 'LED Full HD, ideal para escritório', 1100.00, 9, 90,
'Imagem nítida e rápida', DATE_SUB(CURDATE(), INTERVAL 354 DAY)),
( 'PROD021', 'CAT001', 'EST001', 'Tablet X10', 'Tablet 10 polegadas', 800.00, 50, 30, 'Ótimo produto',
```

```

DATE_SUB(CURDATE(), INTERVAL 180 DAY)),
('PROD022', 'CAT001', 'EST002', 'Smartwatch Z5', 'Relógio inteligente', 600.00, 40, 20, 'Muito bom',
DATE_SUB(CURDATE(), INTERVAL 320 DAY)),
('PROD023', 'CAT001', 'EST003', 'Câmera Digital', 'Câmera 20MP', 1200.00, 25, 15, 'Excelente',
DATE_SUB(CURDATE(), INTERVAL 450 DAY)),
('PROD024', 'CAT001', 'EST004', 'Drone Pro', 'Drone com câmera 4K', 2500.00, 15, 10, 'Incrível',
DATE_SUB(CURDATE(), INTERVAL 210 DAY)),
('PROD025', 'CAT002', 'EST005', 'Jaqueta Jeans', 'Jaqueta masculina', 180.00, 60, 40, 'Estilosa',
DATE_SUB(CURDATE(), INTERVAL 95 DAY)),
('PROD026', 'CAT002', 'EST006', 'Shorts Esportivo', 'Shorts para corrida', 55.00, 100, 80, 'Confortável',
DATE_SUB(CURDATE(), INTERVAL 140 DAY)),
('PROD027', 'CAT002', 'EST007', 'Vestido Floral', 'Vestido feminino', 120.00, 70, 50, 'Lindo',
DATE_SUB(CURDATE(), INTERVAL 275 DAY)),
('PROD028', 'CAT002', 'EST008', 'Moletom Básico', 'Moletom com capuz', 95.00, 80, 60, 'Quentinho',
DATE_SUB(CURDATE(), INTERVAL 380 DAY));

```

```

INSERT INTO pedidos (ID_Pedidos, ID_Conta, data_pedido, status_pedido) VALUES

```

```

('PED001', 1, '2025-10-01', 0),
('PED002', 2, '2025-10-02', 1),
('PED003', 3, '2025-10-03', 0),
('PED004', 4, '2025-10-04', 1),
('PED005', 5, '2025-10-05', 0),
('PED006', 6, '2025-10-06', 1),
('PED007', 7, '2025-10-07', 0),
('PED008', 8, '2025-10-08', 1),
('PED009', 9, '2025-10-09', 0),
('PED010', 10, '2025-10-10', 1),
('PED011', 1, '2025-10-11', 0),
('PED012', 2, '2025-10-12', 1),
('PED013', 3, '2025-10-13', 0),
('PED014', 4, '2025-10-14', 1),
('PED015', 5, '2025-10-15', 0),
('PED016', 6, '2025-10-16', 1),
('PED017', 7, '2025-10-17', 0),
('PED018', 8, '2025-10-18', 1),
('PED019', 9, '2025-10-19', 0),
('PED020', 10, '2025-10-20', 1);

```

```

INSERT INTO pagamento (ID_Pagamento, ID_Pedidos, valor, forma_pagamento, status_pagamento,
data_pagamento) VALUES

```

```

('PAY001', 'PED001', 150.00, 'Cartão de Crédito', 1, '2025-10-02'),
('PAY002', 'PED002', 75.50, 'Boleto Bancário', 1, '2025-10-03'),
('PAY003', 'PED003', 320.00, 'Pix', 0, '2025-10-04'),
('PAY004', 'PED004', 210.00, 'Cartão de Crédito', 1, '2025-10-05'),
('PAY005', 'PED005', 100.00, 'Boleto Bancário', 0, '2025-10-06'),

```



```
( 'PAY006', 'PED006', 450.00, 'Pix', 1, '2025-10-07'),
( 'PAY007', 'PED007', 120.00, 'Cartão de Crédito', 0, '2025-10-08'),
( 'PAY008', 'PED008', 180.00, 'Boleto Bancário', 1, '2025-10-09'),
( 'PAY009', 'PED009', 90.00, 'Pix', 0, '2025-10-10'),
( 'PAY010', 'PED010', 540.00, 'Cartão de Crédito', 1, '2025-10-11'),
( 'PAY011', 'PED011', 160.00, 'Boleto Bancário', 0, '2025-10-12'),
( 'PAY012', 'PED012', 240.00, 'Cartão de Crédito', 1, '2025-10-13'),
( 'PAY013', 'PED013', 70.00, 'Pix', 0, '2025-10-14'),
( 'PAY014', 'PED014', 280.00, 'Boleto Bancário', 1, '2025-10-15'),
( 'PAY015', 'PED015', 140.00, 'Cartão de Crédito', 0, '2025-10-16'),
( 'PAY016', 'PED016', 210.00, 'Pix', 1, '2025-10-17'),
( 'PAY017', 'PED017', 375.00, 'Boleto Bancário', 0, '2025-10-18'),
( 'PAY018', 'PED018', 75.00, 'Cartão de Crédito', 1, '2025-10-19'),
( 'PAY019', 'PED019', 300.00, 'Pix', 0, '2025-10-20'),
( 'PAY020', 'PED020', 160.00, 'Boleto Bancário', 1, '2025-10-21');
```

```
INSERT INTO historico_Compras (ID_Historico, ID_Conta, data_pedido, valor_total, forma_pagamento,
status_pedido, data_pagamento, detalhes_produtos, endereco_entrega) VALUES
```

```
( 'HIST001', 1, '2025-10-01', 150.00, 'Cartão de Crédito', 1, '2025-10-02', 'Smartphone A1, Fone Bluetooth',
'Rua das Flores, 123'),
( 'HIST002', 2, '2025-10-02', 75.50, 'Boleto Bancário', 1, '2025-10-03', 'Camiseta Básica', 'Avenida Central,
456'),
( 'HIST003', 3, '2025-10-03', 320.00, 'Pix', 0, NULL, 'Arroz Integral 1kg, Café Torrado 500g', 'Rua do Sol,
789'),
( 'HIST004', 4, '2025-10-04', 210.00, 'Cartão de Crédito', 1, '2025-10-05', 'Livro SQL Básico, Teclado
Mecânico', 'Rua das Acácias, 321'),
( 'HIST005', 5, '2025-10-05', 100.00, 'Boleto Bancário', 0, NULL, 'Mesa de Escritório', 'Avenida Brasil, 101'),
( 'HIST006', 6, '2025-10-06', 450.00, 'Pix', 1, '2025-10-07', 'Cadeira Gamer', 'Rua das Orquídeas, 654'),
( 'HIST007', 7, '2025-10-07', 120.00, 'Cartão de Crédito', 0, NULL, 'Bola de Futebol, Raquete de Tênis',
'Rua do Lazer, 987'),
( 'HIST008', 8, '2025-10-08', 180.00, 'Boleto Bancário', 1, '2025-10-09', 'Quebra-cabeça 1000 peças',
'Avenida das Palmeiras, 135'),
( 'HIST009', 9, '2025-10-09', 90.00, 'Pix', 0, NULL, 'Óleo para Motor 5W-30', 'Rua do Trabalho, 258'),
( 'HIST010', 10, '2025-10-10', 540.00, 'Cartão de Crédito', 1, '2025-10-11', 'Monitor 24", Teclado Mecânico',
'Rua dos Lagos, 369'),
( 'HIST011', 1, '2025-10-11', 160.00, 'Boleto Bancário', 1, '2025-10-12', 'Fone Bluetooth', 'Rua das Flores,
123'),
( 'HIST012', 2, '2025-10-12', 240.00, 'Cartão de Crédito', 0, NULL, 'Camiseta Básica, Calça Jeans',
'Avenida Central, 456'),
( 'HIST013', 3, '2025-10-13', 70.00, 'Pix', 1, '2025-10-14', 'Livro SQL Básico', 'Rua do Sol, 789'),
( 'HIST014', 4, '2025-10-14', 280.00, 'Boleto Bancário', 1, '2025-10-15', 'Mesa de Escritório', 'Rua das
Acácias, 321'),
( 'HIST015', 5, '2025-10-15', 140.00, 'Cartão de Crédito', 0, NULL, 'Cadeira Gamer', 'Avenida Brasil, 101'),
( 'HIST016', 6, '2025-10-16', 210.00, 'Pix', 1, '2025-10-17', 'Shampoo Anticaspa', 'Rua das Orquídeas,
654'),
( 'HIST017', 7, '2025-10-17', 375.00, 'Boleto Bancário', 1, '2025-10-18', 'Raquete de Tênis, Bola de
Futebol', 'Rua do Lazer, 987'),
( 'HIST018', 8, '2025-10-18', 75.00, 'Cartão de Crédito', 0, NULL, 'Óleo para Motor 5W-30', 'Avenida das
Palmeiras, 135'),
( 'HIST019', 9, '2025-10-19', 300.00, 'Pix', 1, '2025-10-20', 'Bateria Automotiva 60Ah', 'Rua do Trabalho,
258'),
( 'HIST020', 10, '2025-10-20', 160.00, 'Boleto Bancário', 1, '2025-10-21', 'Teclado Mecânico', 'Rua dos
Lagos, 369'),
```

```
( 'HIST021', 1, '2025-09-15', 200.00, 'Cartão de Crédito', 1, '2025-09-16', 'Fone Bluetooth', 'Rua das Flores, 123'),
( 'HIST022', 1, '2025-08-20', 150.00, 'Cartão de Crédito', 1, '2025-08-21', 'Teclado Mecânico', 'Rua das Flores, 123'),
( 'HIST023', 1, '2025-07-10', 300.00, 'Cartão de Crédito', 1, '2025-07-11', 'Monitor 24"', 'Rua das Flores, 123'),
( 'HIST024', 1, '2025-06-05', 100.00, 'Cartão de Crédito', 1, '2025-06-06', 'Camiseta Básica', 'Rua das Flores, 123'),
( 'HIST025', 4, '2025-09-10', 180.00, 'Cartão de Crédito', 1, '2025-09-11', 'Livro SQL Básico', 'Rua das Acácias, 321'),
( 'HIST026', 4, '2025-08-15', 220.00, 'Cartão de Crédito', 1, '2025-08-16', 'Arroz Integral', 'Rua das Acácias, 321'),
( 'HIST027', 4, '2025-07-20', 350.00, 'Cartão de Crédito', 1, '2025-07-21', 'Mesa de Escritório', 'Rua das Acácias, 321'),
( 'HIST028', 4, '2025-06-25', 90.00, 'Cartão de Crédito', 1, '2025-06-26', 'Bola de Futebol', 'Rua das Acácias, 321'),
( 'HIST029', 4, '2025-06-25', 90.00, 'Cartão de Crédito', 1, '2025-06-26', 'Bola de Futebol', 'Rua das Acácias, 321');
```

```
INSERT INTO itens_vendidos (CNPJ, ID_Produtos, quantidade_itens, valor_total, data_venda) VALUES
```

```
( '12345678000101', 'PROD001', 2, 3000.00, '2025-10-01'),
( '12345678000101', 'PROD011', 1, 300.00, '2025-10-11'),
( '22345678000102', 'PROD002', 1, 35.00, '2025-10-02'),
( '22345678000102', 'PROD012', 2, 240.00, '2025-10-12'),
( '32345678000103', 'PROD003', 4, 40.00, '2025-10-03'),
( '32345678000103', 'PROD013', 3, 54.00, '2025-10-13'),
( '42345678000104', 'PROD004', 2, 120.00, '2025-10-04'),
( '42345678000104', 'PROD014', 1, 60.00, '2025-10-14'),
( '52345678000105', 'PROD005', 1, 350.00, '2025-10-05'),
( '52345678000105', 'PROD015', 1, 850.00, '2025-10-15'),
( '62345678000106', 'PROD006', 5, 125.00, '2025-10-06'),
( '62345678000106', 'PROD016', 2, 44.00, '2025-10-16'),
( '72345678000107', 'PROD007', 3, 270.00, '2025-10-07'),
( '72345678000107', 'PROD017', 1, 380.00, '2025-10-17'),
( '82345678000108', 'PROD008', 2, 160.00, '2025-10-08'),
( '82345678000108', 'PROD018', 1, 55.00, '2025-10-18'),
( '92345678000109', 'PROD009', 1, 120.00, '2025-10-09'),
( '92345678000109', 'PROD019', 1, 600.00, '2025-10-19'),
( '10345678000110', 'PROD010', 1, 450.00, '2025-10-10'),
( '10345678000110', 'PROD020', 1, 1100.00, '2025-10-20');
```

9 Scripts SQL de Consultas Obrigatórias

9.1 Consultas com JOIN (Mínimo 10)

1 - Listar contas com endereços:

```
SELECT c.nome, c.email, e.rua, e.cidade FROM conta AS c JOIN endereco AS e ON c.ID_Conta = e.ID_Conta;
```

	nome	email	rua	cidade
►	Carlos Silva	carlos.silva@example.com	Rua das Palmeiras	São Paulo
	Ana Pereira	ana.pereira@example.com	Avenida Brasil	Rio de Janeiro
	João Costa	joao.costa@example.com	Rua Sete de Setembro	Belo Horizonte
	Mariana Rocha	mariana.rocha@example.com	Rua da Paz	Curitiba
	Rafael Souza	rafael.souza@example.com	Avenida dos Andradas	Porto Alegre
	Luana Almeida	luana.almeida@example.com	Rua Joaquim Nabuco	Fortaleza
	Pedro Martins	pedro.martins@example.com	Rua XV de Novembro	Salvador
	Patrícia Oliveira	patricia.oliveira@example.com	Praça da Liberdade	Brasília
	Lucas Pereira	lucas.pereira@example.com	Rua dos Três Irmãos	Natal
	Sofia Costa	sofia.costa@example.com	Rua Dr. José de Alencar	Recife
	Carlos Lima	carlos.lima@example.com	Avenida dos Oceanos	São Luís
	Bárbara Ferreira	barbara.ferreira@example.com	Rua do Sol	Maceió

✓ 141 23:06:40 SELECT c.nome, c.email, e.rua, e.cidade FROM conta AS c JOIN endereco AS e ON c.ID_Conta = e.ID_Con... 20 row(s) returned

Essa consulta com JOIN é útil para localizar endereços físicos para cada usuário presente no banco de dados, facilitando também a entrega dos produtos no destino.

2 – Produtos com categorias:

```
SELECT p.nome AS produto, c.nome AS categoria FROM produtos AS p JOIN categoria AS c ON p.ID_Categoria = c.ID_Categoria;
```

Result Grid		Filter Rows:
	produto	categoria
▶	Smartphone A1	Eletrônicos
	Camiseta Básica	Roupas
	Arroz Integral 1kg	Alimentos
	Livro SQL Básico	Livros
	Mesa de Escritório	Móveis
	Creme Hidratante	Beleza
	Bola de Futebol	Esportes
	Quebra-cabeça 1000 peças	Brinquedos
	Óleo para Motor 5W-30	Automóveis
	Teclado Mecânico	Informática
	Fone Bluetooth	Eletrônicos
	Calça Jeans	Roupas

142 23:06:57 SELECT p.nome AS produto, c.nome AS categoria FROM produtos AS p JOIN categoria AS c ON p.ID_Categ... 20 row(s) returned

Essa consulta com JOIN é útil para dividir os produtos em diferentes categorias, facilitando também a procura dos consumidores no sistema online.

3 – Estoques com suas lojas:

SELECT e.ID_Estoque, l.CNPJ, l.telefone FROM estoque AS e JOIN loja AS l ON e.CNPJ = l.CNPJ;

	ID_Estoque	CNPJ	telefone
▶	EST010	10345678000110	11999990010
	EST011	11345678000111	11999990011
	EST001	12345678000101	11999990001
	EST012	12345678000112	11999990012
	EST013	13345678000113	11999990013
	EST014	14345678000114	11999990014
	EST015	15345678000115	11999990015
	EST016	16345678000116	11999990016
	EST017	17345678000117	11999990017
	EST018	18345678000118	11999990018
	EST019	19345678000119	11999990019
	EST020	20345678000120	11999990020
	EST002	22345678000102	11999990002

143 23:07:22 SELECT e.ID_Estoque, l.CNPJ, l.telefone FROM estoque AS e JOIN loja AS l ON e.CNPJ = l.CNPJ LIMIT 0, 1... 20 row(s) returned

Essa consulta com JOIN é necessária para garantir que há produtos disponíveis no estoque do vendedor antes da venda.

4 – Pedidos com dados na conta:

```
SELECT p.ID_Pedidos, c.nome, p.data_pedido, p.status_pedido FROM pedidos AS p JOIN conta AS c
ON p.ID_Conta = c.ID_Conta;
```

	ID_Pedidos	nome	data_pedido	status_pedido
▶	PED001	Carlos Silva	2025-10-01	0
	PED002	Ana Pereira	2025-10-02	1
	PED003	João Costa	2025-10-03	0
	PED004	Mariana Rocha	2025-10-04	1
	PED005	Rafael Souza	2025-10-05	0
	PED006	Luana Almeida	2025-10-06	1
	PED007	Pedro Martins	2025-10-07	0
	PED008	Patrícia Oliveira	2025-10-08	1
	PED009	Lucas Pereira	2025-10-09	0
	PED010	Sofia Costa	2025-10-10	1
	PED011	Carlos Silva	2025-10-11	0
	PED012	Ana Pereira	2025-10-12	1
	PED013	João Costa	2025-10-13	0

144 23:07:50 SELECT p.ID_Pedidos, c.nome, p.data_pedido, p.status_pedido FROM pedidos AS p JOIN conta AS c ON p.I... 20 row(s) returned

Essa consulta com JOIN é importante para que seja possível identificar o comprador.

5 – Pagamento com pedido:

```
SELECT pay.ID_Pagamento, pay.valor, p.ID_Pedidos, p.status_pedido FROM pagamento
AS pay JOIN pedidos AS p ON pay.ID_Pedidos = p.ID_Pedidos;
```

	ID_Pagamento	valor	ID_Pedidos	status_pedido
▶	PAY001	150.00	PED001	0
	PAY002	75.50	PED002	1
	PAY003	320.00	PED003	0
	PAY004	210.00	PED004	1
	PAY005	100.00	PED005	0
	PAY006	450.00	PED006	1
	PAY007	120.00	PED007	0
	PAY008	180.00	PED008	1
	PAY009	90.00	PED009	0
	PAY010	540.00	PED010	1
	PAY011	160.00	PED011	0
	PAY012	240.00	PED012	1
	PAY013	75.00	PED013	0

145 23:08:16 SELECT pay.ID_Pagamento, pay.valor, p.ID_Pedidos, p.status_pedido FROM pagamento AS pay JOIN pedid... 20 row(s) returned

Essa query é útil porque permite associar pagamentos aos pedidos correspondentes, mostrando o valor pago e o status do pedido, ajudando a empresa a monitorar transações, identificar pagamentos

pendentes e garantir que os pedidos sejam processados corretamente.

6 -- Histórico de compras com usuário:

```
SELECT h.ID_Historico, c.nome, h.valor_total, h.status_pedido FROM historico_compras AS h JOIN conta AS c ON h.ID_Conta = c.ID_Conta;
```

	ID_Historico	nome	valor_total	status_pedido
▶	HIST001	Carlos Silva	150.00	1
	HIST002	Ana Pereira	75.50	1
	HIST003	João Costa	320.00	0
	HIST004	Mariana Rocha	210.00	1
	HIST005	Rafael Souza	100.00	0
	HIST006	Luana Almeida	450.00	1
	HIST007	Pedro Martins	120.00	0
	HIST008	Patrícia Oliveira	180.00	1
	HIST009	Lucas Pereira	90.00	0
	HIST010	Sofia Costa	540.00	1
	HIST011	Carlos Silva	160.00	1
	HIST012	Ana Pereira	240.00	0
	HIST013	Thiago Costa	70.00	1

146 23:08:41 SELECT h.ID_Historico, c.nome, h.valor_total, h.status_pedido FROM historico_compras AS h JOIN conta AS c ON h.ID_Conta = c.ID_Conta; 20 row(s) returned

Essa query é útil porque permite visualizar o histórico de compras de cada cliente, incluindo o valor total e o status do pedido, ajudando a empresa a analisar comportamento de compra, gerar relatórios de vendas e tomar decisões sobre marketing e fidelização.

7 – Carrinho com dados de usuário:

```
SELECT carr.ID_Carrinho, c.nome, carr.quantidade, carr.preco_total FROM carrinho AS carr JOIN conta AS c ON carr.ID_Conta = c.ID_Conta;
```

	ID_Carrinho	nome	quantidade	preco_total
▶	1	Carlos Silva	2	150.00
	2	Ana Pereira	1	75.50
	3	João Costa	4	320.00
	4	Mariana Rocha	3	210.00
	5	Rafael Souza	1	100.00
	6	Luana Almeida	5	450.00
	7	Pedro Martins	2	120.00
	8	Patrícia Oliveira	3	180.00
	9	Lucas Pereira	1	90.00
	10	Sofia Costa	6	540.00
	11	Carlos Lima	2	160.00
	12	Bárbara Ferreira	3	240.00
	13	Thiago Costa	1	70.00

147 23:09:08 SELECT carr.ID_Carrinho, c.nome, carr.quantidade, carr.preco_total FROM carrinho AS carr JOIN conta AS c ON carr.ID_Conta = c.ID_Conta; 20 row(s) returned

Essa query é útil porque permite visualizar o conteúdo dos carrinhos de cada cliente, mostrando a

quantidade de produtos e o valor total, ajudando a empresa a acompanhar compras em andamento, identificar clientes ativos e planejar promoções ou estratégias de venda.

8 – Produtos e quantidade disponível no estoque:

```
SELECT nome, estoque FROM produtos;
```

	nome	estoque
▶	Smartphone A1	100
	Camiseta Básica	200
	Arroz Integral 1kg	500
	Livro SQL Básico	80
	Mesa de Escritório	30
	Creme Hidratante	120
	Bola de Futebol	60
	Quebra-cabeça 1000 peças	70
	Óleo para Motor 5W-30	100
	Teclado Mecânico	40
	Fone Bluetooth	70
	Calça Jeans	90
	Café Torrado 500g	200

148 23:09:35 SELECT nome, estoque FROM produtos LIMIT 0, 10000 20 row(s) returned

Essa query é útil porque permite ver rapidamente o estoque disponível de cada produto, ajudando a empresa a monitorar níveis de estoque, planejar reposições e evitar falta ou excesso de produtos.

9 – Loja com endereço:

```
SELECT l.CNPJ, e.rua, e.numero FROM loja l JOIN endereco e ON l.ID_Endereco = e.ID_Endereco;
```

	CNPJ	rua	numero
▶	12345678000101	Rua das Palmeiras	123
	22345678000102	Avenida Brasil	456
	32345678000103	Rua Sete de Setembro	789
	42345678000104	Rua da Paz	101
	52345678000105	Avenida dos Andradas	202
	62345678000106	Rua Joaquim Nabuco	303
	72345678000107	Rua XV de Novembro	404
	82345678000108	Praça da Liberdade	505
	92345678000109	Rua dos Três Irmãos	606
	10345678000110	Rua Dr. José de Alencar	707
	11345678000111	Avenida dos Oceanos	808
	12345678000112	Rua do Sol	909

149 23:09:57 SELECT l.CNPJ, e.rua, e.numero FROM loja l JOIN endereco e ON l.ID_Endereco = e.ID_Endereco LIMIT 0, ... 20 row(s) returned

Essa query é útil porque permite obter rapidamente o endereço de cada loja, ajudando a empresa a organizar entregas, visitas ou logística, além de manter os registros de localização atualizados.

10 – Produtos, categoria, estoque e loja:

```
SELECT p.nome AS produto, c.nome AS categoria, e.ID_Estoque, l.CNPJ AS loja FROM
produtos AS p JOIN categoria AS c ON p.ID_Categoria = c.ID_Categoria JOIN estoque AS e
ON p.ID_Estoque = e.ID_Estoque JOIN loja AS l ON e.CNPJ = l.CNPJ;
```

	produto	categoria	ID_Estoque	loja
▶	Smartphone A1	Eletrônicos	EST001	12345678000101
	Camiseta Básica	Roupas	EST002	22345678000102
	Arroz Integral 1kg	Alimentos	EST003	32345678000103
	Livro SQL Básico	Livros	EST004	42345678000104
	Mesa de Escritório	Móveis	EST005	52345678000105
	Creme Hidratante	Beleza	EST006	62345678000106
	Bola de Futebol	Esportes	EST007	72345678000107
	Quebra-cabeça 1000 peças	Brinquedos	EST008	82345678000108
	Óleo para Motor 5W-30	Automóveis	EST009	92345678000109
	Tecclado Mecânico	Informática	EST010	10345678000110
	Fone Bluetooth	Eletrônicos	EST001	12345678000101
	Calça Jeans	Roupas	EST002	22345678000102
	Café Torrado 500g	Alimentos	EST003	32345678000103

150 23:10:23 SELECT p.nome AS produto, c.nome AS categoria, e.ID_Estoque, l.CNPJ AS loja FROM produtos AS p JOIN ... 20 row(s) returned

Essa query é útil porque permite visualizar quais produtos estão disponíveis em cada loja, incluindo a categoria e o estoque correspondente, ajudando a empresa a gerenciar inventário, planejar reposição de produtos e analisar a distribuição de itens por loja e categoria.

9.2 Consultas com GROUP BY e HAVING (Mínimo 5)

```
1 -- SELECT CNPJ, COUNT(*) AS total_vendas FROM itens_vendidos group by cnpj;
```



```

1 • SELECT CNPJ, COUNT(*) AS total_vendas
2
3 FROM itens_vendidos
4
5 group by cnpj;

```

Result Grid		
Filter Rows:		
Export:		
	CNPJ	total_vendas
▶	10345678000110	2
	12345678000101	2
	22345678000102	2
	32345678000103	2
	42345678000104	2

Descrição: Essa query é útil porque permite contar quantas vendas cada loja realizou, ajudando a empresa a avaliar o desempenho de cada unidade, identificar as mais ativas e tomar decisões estratégicas sobre marketing, estoque e gestão de vendas.

2 -- SELECT COUNT(ID_Conta) AS contas_criadas , YEAR(dataCadastro) AS ano FROM conta GROUP BY YEAR(dataCadastro);

```

1 • SELECT count(ID_Conta) as contas_criadas ,
2 year(dataCadastro) as ano from conta GROUP BY year(dataCadastro);

```

Result Grid		
Filter Rows:		
Export:		
Wrap Cell Content:		
	contas_criadas	ano
▶	15	2025
	5	2024

Descrição: Essa query é útil porque permite ver quantas contas foram criadas por ano, ajudando a empresa a acompanhar o crescimento de clientes ao longo do tempo, identificar tendências de cadastro e planejar estratégias de marketing ou expansão.

3 -- SELECT COUNT(ID_Produtos) AS quantidade_de_produtos, ID_Categoria FROM produtos

GROUP BY ID_Categoria;

```

1 • SELECT count(ID_Produtos) as quantidade_de_produtos,
2   ID_Categoria from produtos GROUP BY ID_Categoria;

```

quantidade_de_produtos	ID_Categoria
2	CAT001
2	CAT002
2	CAT003
2	CAT004
2	CAT005

Descrição: Essa query é útil porque permite saber quantos produtos existem em cada categoria, ajudando a empresa a avaliar a distribuição de produtos, identificar categorias com mais ou menos oferta e tomar decisões sobre estoque, diversificação e estratégias de vendas.

4 – SELECT (ID_Pedido) AS pedidos_realizados, quantidade AS quantidade_de_produtos FROM carrinho GROUP BY quantidade;

```

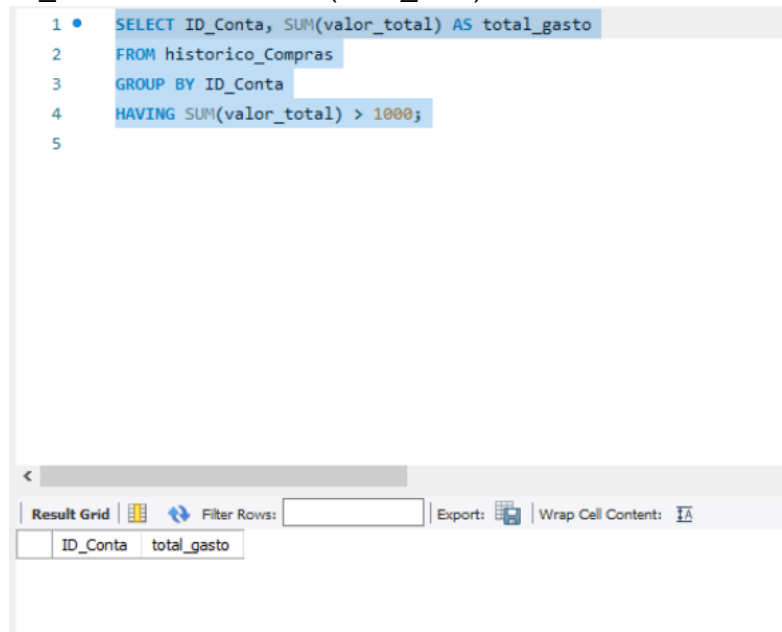
1 • SELECT count(ID_Pedido) as pedidos_realizados,
2   quantidade as quantidade_de_produtos FROM carrinho GROUP BY quantidade;

```

pedidos_realizados	quantidade_de_produtos
5	2
5	1
3	4
4	3
2	5

Descrição: Essa query é útil porque permite analisar quantos pedidos possuem uma determinada quantidade de produtos no carrinho, ajudando a empresa a entender o comportamento de compra dos clientes, identificar padrões de pedidos maiores ou menores e planejar promoções ou estratégias de venda por volume.

5 -- SELECT ID_Conta, SUM(valor_total) AS total_gasto FROM historico_Compras GROUP BY ID_Conta HAVING SUM(valor_total) > 1000;

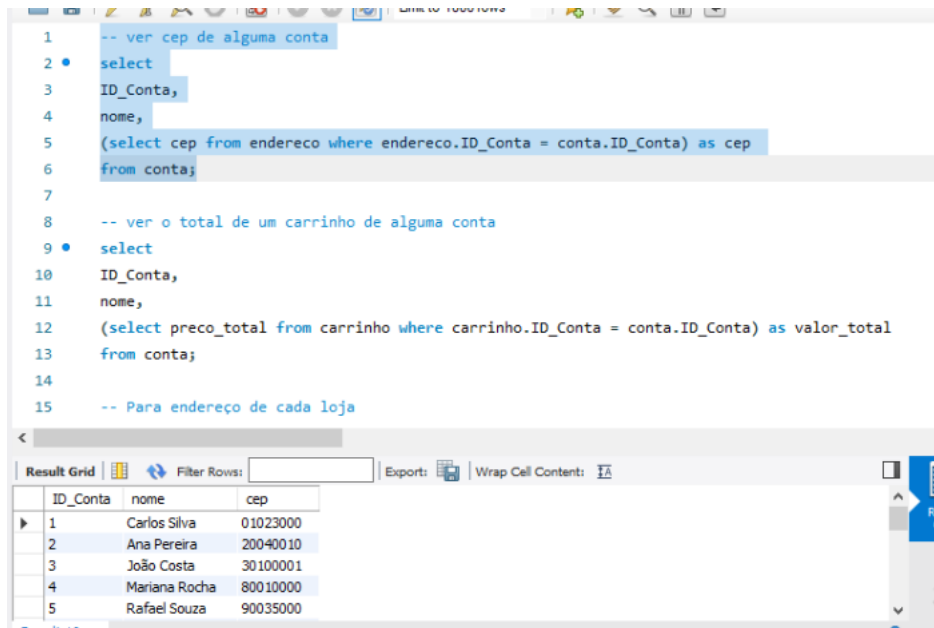


Descrição: Essa query ajuda a identificar clientes premium ou de alto valor, permitindo criar programas de fidelidade, promoções exclusivas ou ofertas personalizadas.

9.3 Subqueries

1 -- Conferir CEP de alguma conta:

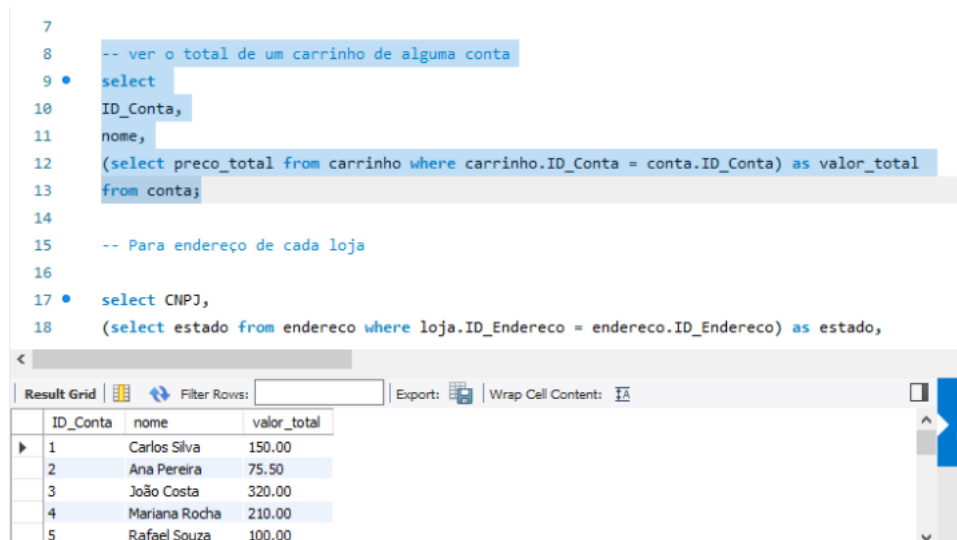
```
SELECT ID_Conta, nome, (SELECT cep FROM endereco WHERE endereco.ID_Conta =
conta.ID_Conta) AS cep FROM conta;
```



Descrição: Essa query ajuda no dia a dia da empresa porque permite consultar rapidamente o CEP associado a cada cliente, facilitando processos como envio de pedidos, cálculo de frete ou logística de entrega.

2 -- Conferir o total de um carrinho de alguma conta:

```
SELECT ID_Conta, nome, (SELECT preco_total FROM carrinho WHERE
carrinho.ID_Conta = conta.ID_Conta) AS valor_total FROM conta;
```



Descrição: Essa query é útil porque permite ver quanto cada cliente tem no carrinho, ajudando a analisar compras em andamento, planejar promoções ou identificar clientes com maior potencial de compra.

3 -- Endereço por loja:

```
SELECT CNPJ, (SELECT estado FROM endereco WHERE loja.ID_Endereco =
endereco.ID_Endereco) AS estado, (SELECT cidade FROM endereco WHERE
```

loja.ID_Endereco = endereco.ID_Endereco) AS cidade, (SELECT rua FROM endereco WHERE loja.ID_Endereco = endereco.ID_Endereco) AS rua, (SELECT numero FROM endereco WHERE loja.ID_Endereco = endereco.ID_Endereco) AS numero FROM loja;

```

13  from conta;
14
15  -- Para endereço de cada loja
16
17  • select CNPJ,
18    (select estado from endereco where loja.ID_Endereco = endereco.ID_Endereco) as estado,
19    (select cidade from endereco where loja.ID_Endereco = endereco.ID_Endereco) as cidade,
20    (select rua from endereco where loja.ID_Endereco = endereco.ID_Endereco) as rua,
21    (select numero from endereco where loja.ID_Endereco = endereco.ID_Endereco) as numero
22  from loja;
23
24  -- Valor de um produto por alguma loja
25  • select CNPJ,
26    ID_Produtos,
27    (select nome from produtos where itens_vendidos.ID_Produtos = produtos.ID_Produtos) as produto

```

Result Grid

CNPJ	estado	cidade	rua	numero
12345678000101	SP	São Paulo	Rua das Palmeiras	123
22345678000102	RJ	Rio de Janeiro	Avenida Brasil	456
32345678000103	MG	Belo Horizonte	Rua Sete de Setembro	789
42345678000104	PR	Curitiba	Rua da Paz	101
52345678000105	RS	Porto Alegre	Avenida dos Andradas	202

Descrição: Essa query é útil porque permite obter rapidamente o endereço completo de cada loja, o que auxilia a organizar entregas, planejar logística, identificar regiões atendidas e manter registros atualizados para comunicação ou visitas.

4 -- Valor de um produto por alguma loja:

```

SELECT CNPJ, ID_Produtos, (SELECT nome FROM produtos WHERE
itens_vendidos.ID_Produtos = produtos.ID_Produtos) AS produto, (SELECT preco FROM
produtos WHERE itens_vendidos.ID_Produtos = produtos.ID_Produtos) AS
preco_unitario, valor_total AS total_arrecadado FROM itens_vendidos;

```

```

22  from loja;
23
24  -- Valor de um produto por alguma loja
25  • select CNPJ,
26    ID_Produtos,
27    (select nome from produtos where itens_vendidos.ID_Produtos = produtos.ID_Produtos) as produto,
28    (select preco from produtos where itens_vendidos.ID_Produtos = produtos.ID_Produtos) as preco_un,
29    valor_total as total_arrecadado
30  from itens_vendidos;
31
32  -- Clientes que gastaram mais que a média
33  • SELECT nome
34  FROM conta
35  WHERE ID_Conta IN (

```

Result Grid

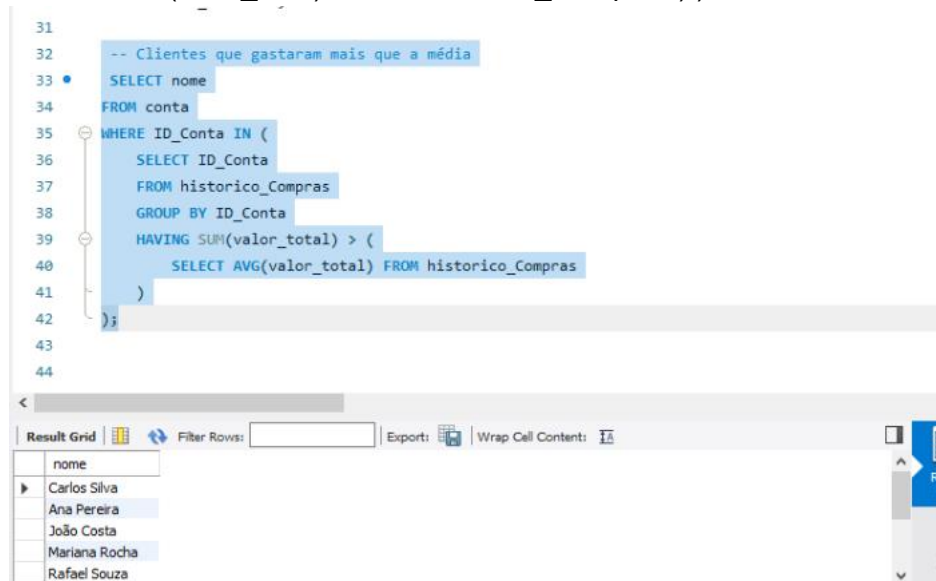
CNPJ	ID_Produtos	produto	preco_unitario	total_arrecadado
12345678000101	PROD001	Smartphone A1	1500.00	3000.00
12345678000101	PROD011	Fone Bluetooth	300.00	300.00
22345678000102	PROD002	Camiseta Básica	35.00	35.00
22345678000102	PROD012	Calça Jeans	120.00	240.00
32345678000103	PROD003	Arroz Integral 1kg	10.00	40.00

Descrição: Essa query é útil porque permite ver quanto cada produto rendeu em vendas por loja, ajudando a analisar faturamento, identificar produtos mais lucrativos e tomar decisões sobre

estoque ou promoções.

5 -- Clientes que gastaram mais que a média:

```
SELECT nome FROM conta WHERE ID_Conta IN ( SELECT ID_Conta FROM
historico_Compras GROUP BY ID_Conta HAVING SUM(valor_total) > ( SELECT
AVG(valor_total) FROM historico_Compras ) );
```



Descrição: Essa query é útil porque permite identificar os clientes que gastam acima da média, ajudando a empresa a focar em clientes mais valiosos, criar campanhas de fidelização ou oferecer promoções personalizadas.

9.4 Criação de VIEWS

1 -- Histórico de compras por cliente:

```
CREATE VIEW view_historico_por_cliente AS SELECT c.nome, h.data_pedido,
h.valor_total, h.forma_pagamento, h.detalhes_produtos FROM historico_Compras h JOIN
conta c ON h.ID_Conta = c.ID_Conta;
```

	nome	data_pedido	valor_total	forma_pagamento	detalhes_produtos
▶	Carlos Silva	2025-10-01	150.00	Cartão de Crédito	Smartphone A1, Fone Bluetooth
	Ana Pereira	2025-10-02	75.50	Boleto Bancário	Camiseta Básica
	João Costa	2025-10-03	320.00	Pix	Arroz Integral 1kg, Café Torrado 500g
	Mariana Rocha	2025-10-04	210.00	Cartão de Crédito	Livro SQL Básico, Teclado Mecânico
	Rafael Souza	2025-10-05	100.00	Boleto Bancário	Mesa de Escritório
	Luana Almeida	2025-10-06	450.00	Pix	Cadeira Gamer
	Pedro Martins	2025-10-07	120.00	Cartão de Crédito	Bola de Futebol, Raquete de Tênis
	Patrícia Oliveira	2025-10-08	180.00	Boleto Bancário	Quebra-cabeça 1000 peças
	Lucas Pereira	2025-10-09	90.00	Pix	Óleo para Motor 5W-30
	Caio Costa	2025-10-10	540.00	Cartão de Crédito	Monitor 24" Teclado Mecânico

125 22:52:03 SELECT * FROM view_historico_por_cliente LIMIT 0, 10000

20 row(s) returned

Descrição: Nesse VIEW, são puxados o nome do comprador, a data do pedido que foi efetuado, o valor pago, o método de pagamento e os detalhes do produto comprado. Essa VIEW é útil para identificar os itens comprados por cada usuário do aplicativo, facilitando a visualização deles e induzindo um algoritmo mais voltado a cada cliente.

2 – Conta dos vendedores:

```
CREATE VIEW view_vendedores AS SELECT ID_Conta, nome, email, telefone,
dataCadastro FROM conta WHERE vendedor = 1;
```

	ID_Conta	nome	email	telefone	dataCadastro
▶	1	Carlos Silva	carlos.silva@example.com	11987654321	2025-10-07 08:30:00
	3	João Costa	joao.costa@example.com	11987654323	2025-09-15 09:15:00
	5	Rafael Souza	rafael.souza@example.com	11987654325	2025-08-20 16:45:00
	7	Pedro Martins	pedro.martins@example.com	11987654327	2025-06-22 18:10:00
	9	Lucas Pereira	lucas.pereira@example.com	11987654329	2025-05-15 12:05:00
	11	Carlos Lima	carlos.lima@example.com	11987654331	2025-03-28 15:15:00
	13	Thiago Gomes	thiago.gomes@example.com	11987654333	2025-02-05 14:45:00
	15	Fernando Dias	fernando.dias@example.com	11987654335	2025-01-10 08:40:00
	17	Ricardo Souza	ricardo.souza@example.com	11987654337	2024-11-30 13:05:00
	19	Guilherme Costa	guilherme.costa@example.com	11987654339	2024-08-05 17:30:00

130 22:56:00 SELECT * FROM view_vendedores LIMIT 0, 10000

10 row(s) returned

Descrição: Nessa VIEW, o banco de dados puxa o nome, o Email, o telefone e a data de cadastro de todos os vendedores com base na booleana vendedor = 1. Essa VIEW é útil para

identificar e separar os vendedores dos não vendedores, facilitando notificações exclusivas e garantindo a veracidade de quem venderá.

3 -- Quantidade de receita gerada por cada tipo de produto vendido:

```
CREATE VIEW view_produtos_mais_vendidos AS SELECT nome, qtd_vendas, preco,
(qtd_vendas * preco) AS receita FROM produtos ORDER BY qtd_vendas DESC;
```

	nome	qtd_vendas	preco	receita
▶	Arroz Integral 1kg	1500	10.00	15000.00
	Café Torrado 500g	800	18.00	14400.00
	Camiseta Básica	500	35.00	17500.00
	Smartphone A1	250	1500.00	375000.00
	Creme Hidratante	220	25.00	5500.00
	Shampoo Anticaspa	210	22.00	4620.00
	Bola de Futebol	180	90.00	16200.00
	Calça Jeans	130	120.00	15600.00
	Fone Bluetooth	110	300.00	33000.00

127 22:53:07 SELECT * FROM view_produtos_mais_vendidos LIMIT 0, 10000

20 row(s) returned

Descrição: Nessa VIEW, é criada uma tabela com o nome dos itens vendidos, a quantidade de vendas, o preço de cada unidade e é feita um produto entre a quantidade de vendas e o preço para gerar a receita (quantidade total recebida). Essa VIEW tem a finalidade de mostrar quais produtos faturam mais, podendo ser utilizada como parâmetro de oferta e demanda em determinados setores.

4 – Pagamentos concluídos com data e valor:

```
CREATE VIEW view_pagamentos_concluidos AS SELECT pg.ID_Pagamento, pg.valor,
pg.forma_pagamento, pg.data_pagamento FROM pagamento pg WHERE
pg.status_pagamento = 1;
```

	ID_Pagamento	valor	forma_pagamento	data_pagamento
▶	PAY001	150.00	Cartão de Crédito	2025-10-02
	PAY002	75.50	Boleto Bancário	2025-10-03
	PAY004	210.00	Cartão de Crédito	2025-10-05
	PAY006	450.00	Pix	2025-10-07
	PAY008	180.00	Boleto Bancário	2025-10-09
	PAY010	540.00	Cartão de Crédito	2025-10-11
	PAY012	240.00	Cartão de Crédito	2025-10-13
	PAY014	280.00	Boleto Bancário	2025-10-15
	PAY016	210.00	Pix	2025-10-17

128 22:53:24 SELECT * FROM view_pagamentos_concluidos LIMIT 0, 10000

11 row(s) returned

Descrição: Nessa VIEW, é criada uma table com o a PK ID_Pagamento, valor pago, a forma de pagamento e a data do pagamento. Essa VIEW é essencial para a garantia de pagamento feita pelo cliente, assegurando que o vendedor não terá prejuízo e que o consumidor tenha uma

nota fiscal de sua compra, garantindo que possa devolver ou reembolsar caso precise.

5 – Pedidos com opiniões registradas:

```
CREATE VIEW view_produtos_opiniones AS SELECT ID_Produtos, nome, opiniones
FROM produtos WHERE opiniones IS NOT NULL AND opiniones <> '';
```

	ID_Produtos	nome	opiniones
▶	PROD001	Smartphone A1	Muito bom, recomendo!
	PROD002	Camiseta Básica	Confortável e durável
	PROD003	Arroz Integral 1kg	Ótima qualidade
	PROD004	Livro SQL Básico	Livro excelente para iniciantes
	PROD005	Mesa de Escritório	Resistente e bonita
	PROD006	Creme Hidratante	Hidratação prolongada
	PROD007	Bola de Futebol	Boa qualidade
	PROD008	Quebra-cabeça 1000 peças	Desafiador e divertido
	PROD009	Óleo para Motor 5W-30	Alta performance
	PROD010	Todas as Manhas	Excelente resposta

129 22:53:39 SELECT * FROM view_produtos_opiniones LIMIT 0, 10000 20 row(s) returned

Descrição: Nessa VIEW, é criada uma table que mostra o ID do produto vendido, o nome e a opinião do usuário a respeito do produto. Essa VIEW serve para facilitar a análise das opiniões de cada produto.

9.5 UPDATES e DELETES Justificados

UPDATES

1 -- Aumentar o preço:

```
UPDATE produtos SET preco = preco * 1.10 WHERE
ID_Categoria = 'CATEGORIA_EXEMPLO';
```

55 22:04:03 UPDATE produtos SET preco = preco * 1.10 WHERE ID_Categoria = 'CATEGORIA_EXEMPLO'

Justificativa: Esse comando é útil porque permite ajustar os preços de todos os produtos de uma categoria de forma rápida, ajudando a empresa a acomodar aumentos de custo, reajustar margens de lucro ou aplicar promoções estratégicas.

2 -- Marcar pedidos como concluídos:

```
UPDATE pedidos SET status_pedido = 1 WHERE ID_Pedidos IN (  
SELECT ID_Pedidos FROM pagamento WHERE status_pagamento  
= 1 );
```

```
✓ 57 22:05:21 UPDATE pedidos SET status_pedido = 1 WHERE ID_Pedidos IN ( SELECT ID_Pedidos FROM pagamento
```

Justificativa: Esse comando é útil porque permite atualizar automaticamente o status dos pedidos que já foram pagos, garantindo que o sistema reflita com precisão quais pedidos estão concluídos, facilitando a gestão de entregas e relatórios de vendas.

3 -- Atualizar telefone:

```
UPDATE conta SET telefone = '41999999999' WHERE ID_Conta = 1;
```

```
✓ 58 22:06:48 UPDATE conta SET telefone = '41999999999' WHERE ID_Conta = 1
```

Justificativa: Esse comando é útil porque permite corrigir ou atualizar rapidamente o telefone de um cliente, garantindo que a empresa tenha contatos atualizados para comunicação, suporte ou envio de notificações sobre pedidos.

DELETE

1 -- Remove pedidos antigos não concluídos:

```
DELETE FROM pedidos WHERE status_pedido  
= 0 AND data_pedido < date_sub(curdate(), interval 6 month) LIMIT 100;
```

```
✓ 59 22:07:14 DELETE FROM pedidos WHERE status_pedido = 0 AND data_pedido < date_sub(curdate(), interval 6 month) LIMIT ...
```

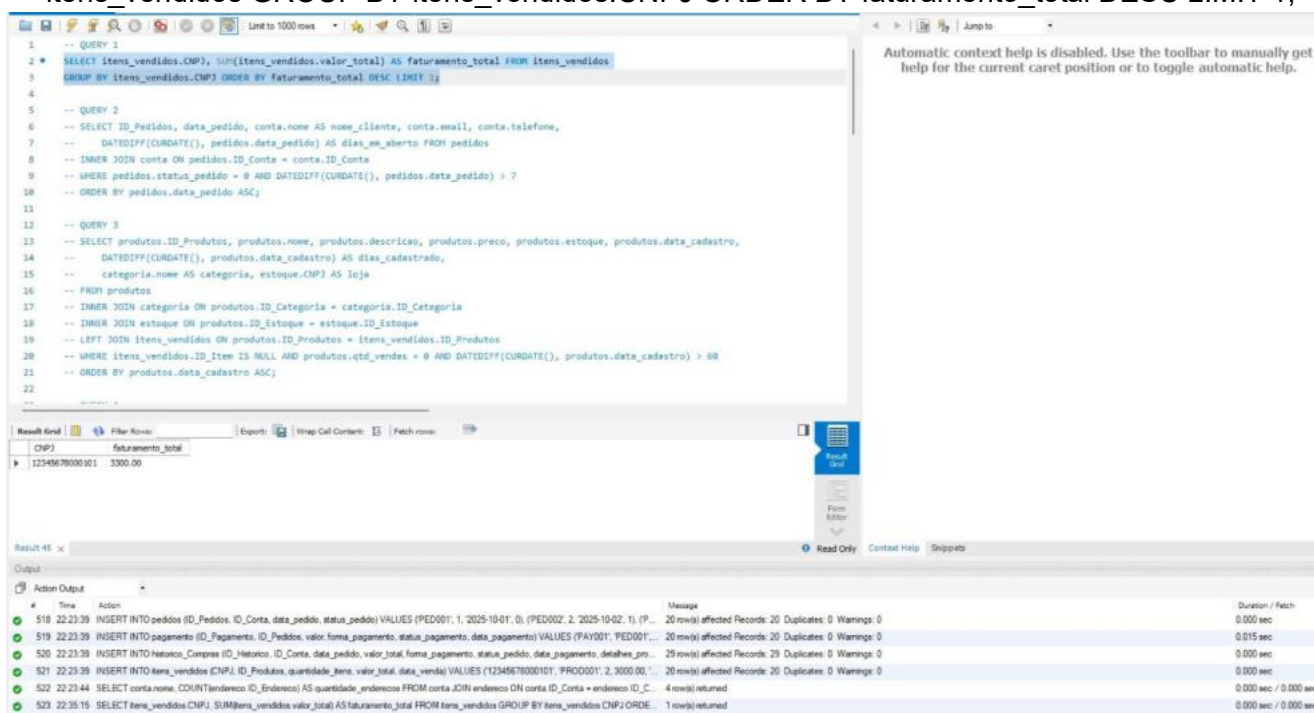
Justificativa: Esse comando é útil porque permite limpar pedidos antigos que não foram concluídos, ajudando a empresa a manter o banco de dados organizado, reduzir dados desnecessários e melhorar o desempenho das consultas.

10 Relatório Explicativo das Consultas Propostas pelo Professor

Apresentar a execução das **10 consultas propostas pelo professor** e as capturas de tela (prints) de cada uma.

1 - Identificar a loja/vendedor com maior faturamento (valor total das vendas confirmadas) no histórico do sistema.

- `SELECT itens_vendidos.CNPJ, SUM(itens_vendidos.valor_total) AS faturamento_total FROM itens_vendidos GROUP BY itens_vendidos.CNPJ ORDER BY faturamento_total DESC LIMIT 1;`



2 - Listar pedidos em aberto (não concluídos) que foram feitos há mais de 7 dias e o nome do cliente que os realizou.

```
SELECT ID_Pedidos, data_pedido, conta.nome AS nome_cliente, conta.email, conta.telefone,
DATEDIFF(CURDATE(), pedidos.data_pedido) AS dias_em_aberto FROM pedidos INNER JOIN conta
ON pedidos.ID_Conta = conta.ID_Conta WHERE pedidos.status_pedido = 0 AND
DATEDIFF(CURDATE(), pedidos.data_pedido) > 7 ORDER BY pedidos.data_pedido ASC;
```

```

6 * SELECT ID_Pedidos, data_pedido, conta.nome AS nome_cliente, conta.email, conta.telefone,
7   DATEDIFF(CURDATE(), pedidos.data_pedido) AS dias_em_aberto FROM pedidos
8   INNER JOIN conta ON pedidos.ID_Conta = conta.ID_Conta
9   WHERE pedidos.status_pedido = 0 AND DATEDIFF(CURDATE(), pedidos.data_pedido) > 60
10  ORDER BY pedidos.data_pedido ASC;
11
12 -- QUERY 3
13 -- SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro,
14 --   DATEDIFF(CURDATE(), produtos.data_cadastro) AS dias_cadastrado,
15 --   categoria.nome AS categoria, estoque.CNPJ AS loja
16 -- FROM produtos
17 -- INNER JOIN categoria ON produtos.ID_Categoria = categoria.ID_Categoria
18 -- INNER JOIN estoque ON produtos.ID_Estoque = estoque.ID_Estoque
19 -- LEFT JOIN itens_vendidos ON produtos.ID_Produtos = itens_vendidos.ID_Produtos
20 -- WHERE (itens_vendidos.ID_Item IS NULL) AND produtos.qtd_vendas = 0 AND DATEDIFF(CURDATE(), produtos.data_cadastro) > 60
21 -- ORDER BY produtos.data_cadastro ASC;
22

```

ID_Pedidos	data_pedido	nome_cliente	email	telefone	dias_em_aberto
PED001	2025-10-01	Carlos Silva	carlos.silva@example.com	11987654321	14
PED003	2025-10-03	Júlia Costa	julia.costa@example.com	11987654323	12
PED005	2025-10-05	Rafael Souza	rafael.souza@example.com	11987654325	10
PED007	2025-10-07	Pedro Martins	pedro.martins@example.com	11987654327	8

3 - Encontrar produtos que nunca foram vendidos (não apareceram em nenhum pedido concluído) e que estão cadastrados há mais de 60 dias.

- SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), produtos.data_cadastro) AS dias_cadastrado, categoria.nome AS categoria, estoque.CNPJ AS loja FROM produtos INNER JOIN categoria ON produtos.ID_Categoria = categoria.ID_Categoria INNER JOIN estoque ON produtos.ID_Estoque = estoque.ID_Estoque WHERE produtos.qtd_vendas = 0 AND DATEDIFF(CURDATE(), produtos.data_cadastro) > 60 ORDER BY produtos.data_cadastro ASC;

```

12 -- QUERY 3
13 -- SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque,
14 --   produtos.data_cadastro,
15 --   DATEDIFF(CURDATE(), produtos.data_cadastro) AS dias_cadastrado,
16 --   categoria.nome AS categoria, estoque.CNPJ AS loja
17 -- FROM produtos
18 -- INNER JOIN categoria ON produtos.ID_Categoria = categoria.ID_Categoria
19 -- INNER JOIN estoque ON produtos.ID_Estoque = estoque.ID_Estoque
20 -- WHERE produtos.qtd_vendas = 0 AND DATEDIFF(CURDATE(), produtos.data_cadastro) > 60
21 -- ORDER BY produtos.data_cadastro ASC;
22
23 -- QUERY 4
24 -- SELECT endereco.estado,
25 --   COUNT(pedidos.ID_Pedidos) AS total_pedidos_concluidos,
26 --   SUM(pagamento.valor) AS valor_total
27 -- FROM endereco
28 -- LEFT JOIN pedidos ON endereco.ID_Endereco = pedidos.ID_Endereco
29 -- LEFT JOIN pagamentos ON pedidos.ID_Pedidos = pagamentos.ID_Pedidos
30 -- GROUP BY endereco.estado;
31

```

ID_Produtos	nome	descricao	preco	estoque	data_cadastro	dias_cadastrado	categoria	loja
PROD008	Quebra-cabeça 1000 peças	Imagem de paisagem natural	80.00	70	2023-12-18 00:00:00	667	Brinquedos	8234567890123
PROD001	Smartphone A1	Smartphone com tela 6.1", câmera 12MP	1300.00	300	2023-02-12 00:00:00	245	Eletrônicos	1234567890123
PROD015	Cadeira Gamer	Ergonômica com ajuste de altura	850.00	25	2023-07-18 00:00:00	89	Móveis	5234567890123

4 - Gerar um relatório com o volume total de pedidos concluídos por Estado do endereço de cadastro do cliente.

- SELECT endereco.estado, COUNT(pedidos.ID_Pedidos) AS total_pedidos_concluidos, SUM(pagamento.valor) AS valor_total FROM pedidos INNER JOIN conta ON pedidos.ID_Conta = conta.ID_Conta INNER JOIN endereco ON conta.ID_Conta = endereco.ID_Conta INNER JOIN pagamento ON pedidos.ID_Pedidos = pagamento.ID_Pedidos WHERE pedidos.status_pedido = 1 GROUP BY endereco.estado ORDER BY total_pedidos_concluidos DESC;

Result 4

estado	total_pedidos_concluidos	valor_total
RJ	2	515.00
PA	2	600.00
CE	2	600.00
DF	2	255.00
PE	2	700.00

Result 5

#	Time	Action	Message	Duration / Fetch
535	22:54:47	SELECT p.ID_Produtos, p.nome, p.data_venda, DATEDIFF(CURDATE(), p.data_cadastro) AS dias FROM produtos p ORDER BY p.data_c...	28 rows(s) returned	0.000 sec / 0.000 sec
536	22:57:20	SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS dias FROM produtos p ORDER BY p.data_c...	0 rows(s) returned	0.000 sec / 0.000 sec
537	22:57:35	SELECT p.ID_Produtos, p.nome, p.descricao, p.preco, p.estoque, p.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS dias FROM produtos p ORDER BY p.data_c...	3 rows(s) returned	0.000 sec / 0.000 sec
538	23:00:08	SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS dias FROM produtos p ORDER BY p.data_c...	3 rows(s) returned	0.000 sec / 0.000 sec
539	23:00:21	SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS dias FROM produtos p ORDER BY p.data_c...	3 rows(s) returned	0.000 sec / 0.000 sec
540	23:03:10	SELECT endereco.estado, COUNT(pedidos.ID_Pedidos) AS total_pedidos_concluidos, SUM(pagamento.valor) AS valor_total FROM pedidos p INNER JOIN conta c ON p.ID_Conta = c.ID_Conta INNER JOIN endereco e ON c.ID_Conta = e.ID_Conta INNER JOIN pagamento pa ON p.ID_Pedidos = pa.ID_Pedidos WHERE p.status_pedido = 1 GROUP BY e.estado ORDER BY total_pedidos_concluidos DESC	5 rows(s) returned	0.000 sec / 0.000 sec

5 - Identificar os 10 produtos mais vendidos no último mês que estão com estoque abaixo de 10 unidades (alerta de reabastecimento).

- SELECT produtos.ID_Produtos, produtos.nome, produtos.preco, produtos.estoque, SUM(itens_vendidos.quantidade_itens) AS total_vendido_mes, SUM(itens_vendidos.valor_total) AS faturamento_mes, categoria.nome AS categoria, estoque.CNPJ AS loja FROM produtos INNER JOIN itens_vendidos ON produtos.ID_Produtos = itens_vendidos.ID_Produtos INNER JOIN categoria ON produtos.ID_Categoria = categoria.ID_Categoria INNER JOIN estoque ON produtos.ID_Estoque = estoque.ID_Estoque WHERE produtos.estoque < 10 AND itens_vendidos.data_venda >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH) GROUP BY produtos.ID_Produtos, produtos.nome, produtos.preco, produtos.estoque, categoria.nome, estoque.CNPJ ORDER BY total_vendido_mes DESC LIMIT 10;

3 -- QUERY 5

4 SELECT produtos.ID_Produto, produtos.nome, produtos.preco, produtos.estoque,

5 SUM(itens_vendidos.quantidade_itens) AS total_vendidos_mes,

6 SUM(itens_vendidos.valor_total) AS faturamento_mes,

7 categoria.nome AS categoria, estoque.CNPJ AS loja

8 FROM produtos

9 INNER JOIN itens_vendidos ON produtos.ID_Produto = itens_vendidos.ID_Produto

10 INNER JOIN categoria ON produtos.ID_Categoria = categoria.ID_Categoria

11 INNER JOIN estoque ON produtos.ID_Estoque = estoque.ID_Estoque

12 WHERE produtos.estoque < 10 AND itens_vendidos.data_venda >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)

13 GROUP BY produtos.ID_Produto, produtos.nome, produtos.preco, produtos.estoque, categoria.nome, estoque.CNPJ

14 ORDER BY total_vendidos_mes DESC LIMIT 10

15

16 -- QUERY 6

17 SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone,

Result Grid

ID_Produto	nome	preco	estoque	total_vendidos_mes	faturamento_mes	categoria	loja
PROD003	Arroz Integral 5kg	10.00	2	4	40.00	Alimentos	32345678900103
PROD007	Bola de Futebol	90.00	3	3	270.00	Esportes	72345678900107
PROD009	Mesa de Esquimó	250.00	2	1	250.00	Hóteis	52345678900109
PROD017	Raquele de Tênis	380.00	4	1	380.00	Esportes	72345678900107
PROD009	Óleo para Motor 5W-30	120.00	5	1	120.00	Automóveis	92345678900109
PROD019	Bateria Automotiva 55Ah	600.00	6	1	600.00	Automóveis	92345678900109
PROD010	Tecido Hecroide	450.00	8	1	450.00	Informática	10345678900110
PROD030	Monitor 24"	1100.00	9	1	1100.00	Informática	10345678900110

Result 63

Output

Action Output

#	Time	Action	Message	Duration / Fetch
536	22:57:20	SELECT produtos.ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS d...	0 row(s) returned	0.000 sec / 0.000 sec
537	22:57:35	SELECT p.ID_Produto, p.nome, p.descricao, p.preco, p.estoque, p.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS d...	3 row(s) returned	0.000 sec / 0.000 sec
538	23:00:08	SELECT produtos.ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
539	23:00:21	SELECT produtos.ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
540	23:03:10	SELECT endereco.estado, COUNT(pedidos.ID_Pedido) AS total_pedidos_concluidos, SUM(pagamento.valor) AS valor_total FROM pedidos INN...	5 row(s) returned	0.000 sec / 0.000 sec
541	23:04:09	SELECT produtos.ID_Produto, produtos.nome, produtos.preco, produtos.estoque, SUM(itens_vendidos.quantidade_itens) AS total_vendidos_mes, ...	8 row(s) returned	0.000 sec / 0.000 sec

6 - Encontrar clientes que utilizaram o 'Cartão de Crédito' como forma de pagamento em pelo menos 5 transações no histórico de compras.

- SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, COUNT(historico_Compras.ID_Historico) AS total_transacoes_cartao, SUM(historico_Compras.valor_total) AS valor_total_gasto FROM conta INNER JOIN historico_Compras ON conta.ID_Conta = historico_Compras.ID_Conta WHERE historico_Compras.forma_pagamento = 'Cartão de Crédito' GROUP BY conta.ID_Conta, conta.nome, conta.email, conta.telefone HAVING COUNT(historico_Compras.ID_Historico) >= 5 ORDER BY total_transacoes_cartao DESC;

3 -- QUERY 6

4 SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone,

5 COUNT(historico_Compras.ID_Historico) AS total_transacoes_cartao,

6 SUM(historico_Compras.valor_total) AS valor_total_gasto

7 FROM conta

8 INNER JOIN historico_Compras ON conta.ID_Conta = historico_Compras.ID_Conta

9 WHERE historico_Compras.forma_pagamento = 'Cartão de Crédito'

10 GROUP BY conta.ID_Conta, conta.nome, conta.email, conta.telefone

11 HAVING COUNT(historico_Compras.ID_Historico) >= 5

12 ORDER BY total_transacoes_cartao DESC

13

14 -- QUERY 7

15 SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao,

16 COUNT(produtos.ID_Produto) AS total_produtos

17 FROM categoria

18 INNER JOIN produtos ON categoria.ID_Categoria = produtos.ID_Categoria

Result Grid

ID_Conta	nome	email	telefone	total_transacoes_cartao	valor_total_gasto
4	Mariana Rocha	mariana.rocha@example.com	11987654324	8	1140.00
1	Carlos Silva	carlos.silva@example.com	11987654321	5	900.00

Result 94

Output

Action Output

#	Time	Action	Message	Duration / Fetch
537	22:57:35	SELECT produtos.ID_Produto, p.nome, p.descricao, p.preco, p.estoque, p.data_cadastro, DATEDIFF(CURDATE(), p.data_cadastro) AS d...	3 row(s) returned	0.000 sec / 0.000 sec
538	23:00:08	SELECT produtos.ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
539	23:00:21	SELECT produtos.ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
540	23:03:10	SELECT endereco.estado, COUNT(pedidos.ID_Pedido) AS total_pedidos_concluidos, SUM(pagamento.valor) AS valor_total FROM pedidos INN...	5 row(s) returned	0.000 sec / 0.000 sec
541	23:04:09	SELECT produtos.ID_Produto, produtos.nome, produtos.preco, produtos.estoque, SUM(itens_vendidos.quantidade_itens) AS total_vendidos_mes, ...	8 row(s) returned	0.000 sec / 0.000 sec
542	23:07:56	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, COUNT(historico_Compras.ID_Historico) AS total_transacoes_cartao, SUM(his...	2 row(s) returned	0.000 sec / 0.000 sec

7 - Listar as categorias que contêm mais de 5 produtos distintos no catálogo.

- SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao, COUNT(produtos.ID_Produtos) AS total_produtos FROM categoria INNER JOIN produtos ON categoria.ID_Categoria = produtos.ID_Categoria GROUP BY categoria.ID_Categoria, categoria.nome, categoria.descricao HAVING COUNT(produtos.ID_Produtos) > 5 ORDER BY total_produtos DESC;

```

-- QUERY 7
4 * SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao,
5     COUNT(produtos.ID_Produtos) AS total_produtos
6 FROM categoria
7 INNER JOIN produtos ON categoria.ID_Categoria = produtos.ID_Categoria
8 GROUP BY categoria.ID_Categoria, categoria.nome, categoria.descricao
9 HAVING COUNT(produtos.ID_Produtos) > 5
10 ORDER BY total_produtos DESC;

-- QUERY 8
13 SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, historico_Compras.ID_Historico,
14     historico_Compras.data_pedido, historico_Compras.valor_total, historico_Compras.forma_pagamento,
15     historico_Compras.detalhes_produtos, DATEDIFF(CURDATE(), historico_Compras.data_pedido) AS dias_em_aberto
16 FROM conta
17 INNER JOIN historico_Compras ON conta.ID_Conta = historico_Compras.ID_Conta

```

ID_Categoria	nome	descricao	total_produtos
CAT001	Eletrônicos	Dispositivos eletrônicos e gadgets	6
CAT002	Roupas	Vestimenta masculina, feminina e infantil	6

Result 03

Output

Action Output

#	Time	Action	Message	Duration / Fetch
530	23:00:08	SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
539	23:00:21	SELECT produtos.ID_Produtos, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CUR...	3 row(s) returned	0.000 sec / 0.000 sec
540	23:03:10	SELECT endereco.estado, COUNT(pedidos.ID_Pedidos) AS total_pedidos_concluidos, SUM(pagamento.valor) AS valor_total FROM pedidos INNER...	5 row(s) returned	0.000 sec / 0.000 sec
541	23:04:09	SELECT produtos.ID_Produtos, produtos.nome, produtos.preco, produtos.estoque, SUM(Mais_vendidos.quantidade_geral) AS total_vendido_mes...	8 row(s) returned	0.000 sec / 0.000 sec
542	23:07:56	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, COUNT(historico_Compras.ID_Historico) AS total_transacoes, contas...	2 row(s) returned	0.000 sec / 0.000 sec
543	23:08:36	SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao, COUNT(produtos.ID_Produtos) AS total_produtos FROM categoria INNER J...	2 row(s) returned	0.000 sec / 0.000 sec

8 - Encontrar clientes com compras registradas no histórico que têm status "em aberto" e sem data de pagamento registrada (auditoria de pedidos).

- SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, historico_Compras.ID_Historico, historico_Compras.data_pedido, historico_Compras.valor_total, historico_Compras.forma_pagamento, historico_Compras.detalhes_produtos, DATEDIFF(CURDATE(), historico_Compras.data_pedido) AS dias_em_aberto FROM conta INNER JOIN historico_Compras ON conta.ID_Conta = historico_Compras.ID_Conta WHERE historico_Compras.status_pedido = 0 AND historico_Compras.data_pagamento IS NULL ORDER BY historico_Compras.data_pedido ASC;

```

3 -- QUERY 8
4 * SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, historico_compras.ID_Historico,
5     historico_compras.data_pedido, historico_compras.valor_total, historico_compras.forma_pagamento,
6     historico_compras.detalhes_produtos, DATEDIFF(CURDATE(), historico_compras.data_pedido) AS dias_em_aberto
7 FROM conta
8 INNER JOIN historico_compras ON conta.ID_Conta = historico_compras.ID_Conta
9 WHERE historico_compras.status_pedido = 0 AND historico_compras.data_pagamento IS NULL
10 ORDER BY historico_compras.data_pedido ASC;
11
12 -- QUERY 9
13 -- SELECT nome, preco
14 -- FROM produtos
15 -- ORDER BY preco DESC
16 -- LIMIT 1;
17
Result Grid
ID_Conta  nome  email  telefone  ID_Historico  data_pedido  valor_total  forma_pagamento  detalhes_produtos  dias_em_aberto
1  Jolo Costa  jolo.costa@example.com  11987654321  HIST003  2025-10-03  320.00  Pix  Amorz Integral 1kg, Café Tornado 500g  12
2  Rafael Souza  rafael.souza@example.com  11987654322  HIST005  2025-10-05  180.00  Boleto Bancário  Pressão de Escalção  10
3  Pedro Martins  pedro.martins@example.com  11987654327  HIST007  2025-10-07  120.00  Cartão de Crédito  Bola de Futebol, Raquete de Tênis  8
4  Lucas Pereira  lucas.pereira@example.com  11987654329  HIST009  2025-10-09  90.00  Pix  Óleo para Motor 5W-30  6
5  Ana Pereira  ana.pereira@example.com  11987654322  HIST012  2025-10-12  240.00  Cartão de Crédito  Camiseta Básica, Calça Jeans  3
6  Rafael Souza  rafael.souza@example.com  11987654323  HIST015  2025-10-15  140.00  Cartão de Crédito  Caderno Gamer  0
7  Patricia Oliveira  patricia.oliveira@example.com  11987654328  HIST018  2025-10-18  75.00  Cartão de Crédito  Óleo para Motor 5W-30  -3
  
```

Output

#	Time	Action	Message	Duration / Fetch
539	23:00:21	SELECT produtos ID_Produto, produtos.nome, produtos.descricao, produtos.preco, produtos.estoque, produtos.data_cadastro, DATEDIFF(CURDATE(), produtos.data_pedido) AS dias_em_aberto	3 rows(s) returned	0.000 sec / 0.000 sec
540	23:03:10	SELECT endereco_estado: COUNT(produtos.ID_Produto) AS total_produtos_concluidos, SUM(pagamentos.valor) AS valor_total FROM produtos INNER JOIN pagamentos ON produtos.ID_Produto = pagamentos.ID_Produto	5 rows(s) returned	0.000 sec / 0.000 sec
541	23:04:09	SELECT produtos ID_Produto, produtos.nome, produtos.preco, produtos.estoque, SUM(historico_compras.quantidade_pedido) AS total_pedido, SUM(historico_compras.valor_total) AS total_valor_pedido	8 rows(s) returned	0.000 sec / 0.000 sec
542	23:07:56	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, COUNT(historico_compras.ID_Historico) AS total_transacoes, SUM(historico_compras.valor_total) AS total_valor_transacoes	2 rows(s) returned	0.000 sec / 0.000 sec
543	23:08:36	SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao, COUNT(produtos.ID_Produto) AS total_produtos FROM categoria INNER JOIN produtos ON categoria.ID_Categoria = produtos.ID_Categoria	2 rows(s) returned	0.000 sec / 0.000 sec
544	23:09:36	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, historico_compras.ID_Historico, historico_compras.data_pedido, historico_compras.valor_total, historico_compras.forma_pagamento, historico_compras.detalhes_produtos, DATEDIFF(CURDATE(), historico_compras.data_pedido) AS dias_em_aberto	7 rows(s) returned	0.000 sec / 0.000 sec

9 - Descobrir qual é o produto com o preço mais elevado no catálogo, mostrando o nome do produto e seu preço.

SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 1;

```

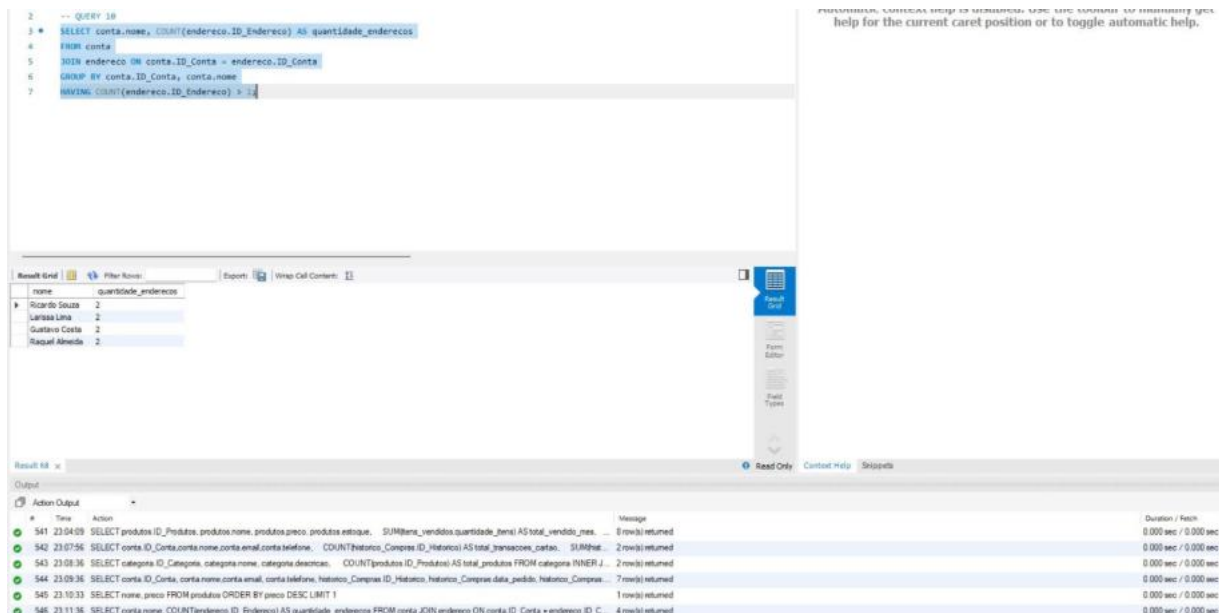
3 -- QUERY 9
4 * SELECT nome, preco
5 FROM produtos
6 ORDER BY preco DESC
7 LIMIT 1;
8
9 -- QUERY 10
10 -- SELECT conta.nome, COUNT(endereco.ID_Endereco) AS quantidade_enderecos
11 -- FROM conta
12 -- JOIN endereco ON conta.ID_Conta = endereco.ID_Conta
13 -- GROUP BY conta.ID_Conta, conta.nome
14 -- HAVING COUNT(endereco.ID_Endereco) > 1;
15
Result Grid
nome  preco
1  Drone Pro  2500.00
  
```

Output

#	Time	Action	Message	Duration / Fetch
540	23:03:10	SELECT endereco_estado: COUNT(produtos.ID_Produto) AS total_produtos_concluidos, SUM(pagamentos.valor) AS valor_total FROM produtos INNER JOIN pagamentos ON produtos.ID_Produto = pagamentos.ID_Produto	5 rows(s) returned	0.000 sec / 0.000 sec
541	23:04:09	SELECT produtos ID_Produto, produtos.nome, produtos.preco, produtos.estoque, SUM(historico_compras.quantidade_pedido) AS total_pedido, SUM(historico_compras.valor_total) AS total_valor_pedido	8 rows(s) returned	0.000 sec / 0.000 sec
542	23:07:56	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, COUNT(historico_compras.ID_Historico) AS total_transacoes, SUM(historico_compras.valor_total) AS total_valor_transacoes	2 rows(s) returned	0.000 sec / 0.000 sec
543	23:08:36	SELECT categoria.ID_Categoria, categoria.nome, categoria.descricao, COUNT(produtos.ID_Produto) AS total_produtos FROM categoria INNER JOIN produtos ON categoria.ID_Categoria = produtos.ID_Categoria	2 rows(s) returned	0.000 sec / 0.000 sec
544	23:09:36	SELECT conta.ID_Conta, conta.nome, conta.email, conta.telefone, historico_compras.ID_Historico, historico_compras.data_pedido, historico_compras.valor_total, historico_compras.forma_pagamento, historico_compras.detalhes_produtos, DATEDIFF(CURDATE(), historico_compras.data_pedido) AS dias_em_aberto	7 rows(s) returned	0.000 sec / 0.000 sec
545	23:10:33	SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 1	1 row(s) returned	0.000 sec / 0.000 sec

10 - Encontrar clientes que possuem mais de um endereço cadastrado no sistema, listando o nome e a contagem de endereços.

- SELECT conta.nome, COUNT(endereco.ID_Endereco) AS quantidade_enderecos FROM conta JOIN endereco ON conta.ID_Conta = endereco.ID_Conta GROUP BY conta.ID_Conta, conta.nome HAVING COUNT(endereco.ID_Endereco) > 1;



10.1 Ajustes e Melhorias na Modelagem (Revisão da Parte 1)

Foi adicionada uma nova tabela para controlar a quantidade de produtos vendidos por cada loja. Essa melhoria permite que a empresa tenha um acompanhamento detalhado das vendas por produto e por loja, facilitando a análise de desempenho, a identificação de produtos mais vendidos e menos vendidos, além de apoiar decisões estratégicas sobre reposição de estoque, promoções e planejamento de vendas.

- Implementação do banco de dados com dados reais ou simulados, contendo ao menos 20 registros por tabela;
- Script SQL com consultas aplicáveis ao cenário proposto, contendo no mínimo:
 - 10 consultas com JOIN (incluindo pelo menos 1 com 3 tabelas);
 - 5 consultas com GROUP BY e HAVING;
 - 5 SUBQUERY;
 - 5 criações de VIEW útil ao contexto;
 - 3 UPDATE e 1 DELETE com cláusulas bem justificadas.
- Execução de 10 consultas propostas pelo professor, aplicadas ao banco do projeto;
- Relatório explicativo em PDF contendo:
 - Capturas de tela (prints) da execução de cada consulta;
 - Breve explicação de como a consulta pode ser utilizada no dia a dia da organização

modelada;

- 1 Seção destacando ajustes e melhorias feitas em relação à Parte 1 (modelagem), com justificativa das decisões.

Parte III

Parte 3: Controle de Acesso, Transações e Otimização (RA3)

11 Segurança e controle de acesso

11.1 Criação de Usuários e Perfis de Acesso (DCL)

Implemente, via script SQL (DCL), a criação de pelo menos 3 perfis de acesso diferentes no banco, justificando os privilégios concedidos a cada um.

- **Perfil 1: Cliente**

```
GRANT SELECT ON TAGH.produto TO 'cliente'@'TAGHwebcommerce.com.br';  
GRANT INSERT ON TAGH.pedidos TO 'cliente'@'TAGHwebcommerce.com.br';  
GRANT SELECT ON TAGH.historico TO 'cliente'@'TAGHwebcommerce.com.br';
```

Justificativa: Permite consultar produtos (SELECT), adicionar pedidos (INSERT) e ver o histórico de compras (SELECT).

- **Perfil 2: Loja**

```
GRANT SELECT, UPDATE ON TAGH.produto TO 'loja'@'TAGHwebcommerce.com.br';  
GRANT SELECT, INSERT, UPDATE ON TAGH.pedidos TO 'loja'@'TAGHwebcommerce.com.br';  
GRANT SELECT ON TAGH.pagamento TO 'loja'@'TAGHwebcommerce.com.br';
```

Justificativa: Permite consultar e atualizar produtos (SELECT, UPDATE), consultar, inserir e atualizar pedidos (SELECT, INSERT, UPDATE) e ver os pagamentos realizados (SELECT).

- **Perfil 3: Admin**

```
GRANT ALL PRIVILEGES ON TAGH.* TO 'admin'@'TAGHwebcommerce.com.br' WITH GRANT  
OPTION;
```

Justificativa: Acesso total (ALL PRIVILEGES).

11.2 Simulação de Revogação de Acesso

Simule a revogação de um privilégio importante de um dos perfis, demonstrando o script REVOKE e explicando o cenário de segurança que justifica a ação.

```
REVOKE UPDATE ON TAGH.pedido FROM 'loja'@'TAGHwebcommerce.com.br';
```

Justificativa de Segurança: A Loja não deve ter permissão para alterar pedidos após o fechamento da venda, prevenindo fraudes, manipulação de valores ou alterações indevidas no histórico de venda dos clientes.

12 Controle de Transações (Commit/Rollback)

12.1 Simulação de Transação de Venda (Sucesso)

START TRANSACTION; -- 1. Insere o novo pedido e produtos no histórico

```
INSERT INTO historico_compras (ID_Historico, ID_Conta, data_pedido, valor_total,
forma_pagamento,status_pedido,detalhes_produtos,endereco_entrega) VALUES ('HIST021', 11,
NOW(),1500,'PIX','0','Smartphone A1', 'Rua aleatória,517');
```

-- 2. Atualiza o estoque

```
UPDATE produtos SET estoque = estoque - 1 WHERE ID_Produtos = 'PROD001'; COMMIT;
```

Justificativa: No primeiro passo incluímos todos os dados do pedido e da compra, menos a data de pagamento devido ao pedido exemplo ser um cenário de pagamento ainda não realizado, no segundo atualizamos o estoque, reduzindo 1 da quantia atual, esse 1 vindo do produto comprado, o pedido só será realizado se o estoque tiver 1 ou mais do produto desejado.

12.2 Simulação de Transação com Falha (Rollback)

START TRANSACTION; -- 1. Insere o novo pedido e produtos no histórico I

```
INSERT INTO historico_compras (ID_Historico, ID_Conta, data_pedido, valor_total,
forma_pagamento,status_pedido,detalhes_produtos,endereco_entrega) VALUES ('HIST0369', 11,
NOW(),1500,'PIX','0','Smartphone A1', 'Rua aleatória,517');
```

-- 2. Atualiza o estoque

```
UPDATE produtos SET estoque = estoque - 1 WHERE ID_Produtos = 'ZROD001';
```

-- Não tem o produto no estoque (registro inexistente) ROLLBACK;

-- Caso de tudo certo -- COMMIT;

Justificativa: A transação agrupa várias operações que devem ocorrer juntas, se acontecer alguma falha, como por exemplo o produto não existir (não disponível no estoque), então o rollback desfaz todas as alterações feitas na transação como o insert no histórico, isso mantém o banco consistente, enquanto o commit seria usado para confirmar as alterações se desse tudo certo.

13 Otimização de Desempenho e Boas Práticas

13.1 Análise de Índices

- **Consulta Escolhida:**

```
SELECT produtos.ID_Produtos, produtos.nome, produtos.preco, produtos.estoque,
SUM(itens_vendidos.quantidade_itens) AS total_vendido_mes, SUM(itens_vendidos.valor_total) AS
faturamento_mes, categoria.nome AS categoria, estoque.CNPJ AS loja FROM produtos INNER JOIN
itens_vendidos ON produtos.ID_Produtos = itens_vendidos.ID_Produtos INNER JOIN categoria ON
produtos.ID_Categoria = categoria.ID_Categoria INNER JOIN estoque ON produtos.ID_Estoque =
estoque.ID_Estoque WHERE produtos.estoque < 10 AND itens_vendidos.data_venda >=
DATE_SUB(CURDATE(), INTERVAL 1 MONTH) GROUP BY produtos.ID_Produtos, produtos.nome,
produtos.preco, produtos.estoque, categoria.nome, estoque.CNPJ ORDER BY total_vendido_mes DESC
LIMIT 10;
```

- **Plano de Execução (EXPLAIN):**

```
EXPLAIN SELECT produtos.ID_Produtos, produtos.nome, produtos.preco, produtos.estoque,
SUM(itens_vendidos.quantidade_itens) AS total_vendido_mes, SUM(itens_vendidos.valor_total) AS
faturamento_mes, categoria.nome AS categoria, estoque.CNPJ AS loja FROM produtos INNER JOIN
itens_vendidos ON produtos.ID_Produtos = itens_vendidos.ID_Produtos INNER JOIN categoria ON
produtos.ID_Categoria = categoria.ID_Categoria INNER JOIN estoque ON produtos.ID_Estoque =
estoque.ID_Estoque WHERE produtos.estoque < 10 AND itens_vendidos.data_venda >=
DATE_SUB(CURDATE(), INTERVAL 1 MONTH) GROUP BY produtos.ID_Produtos, produtos.nome,
produtos.preco, produtos.estoque, categoria.nome, estoque.CNPJ ORDER BY total_vendido_mes DESC
LIMIT 10;
```

- **Proposta de Otimização:**

Melhora o filtro do estoque: *CREATE INDEX idx_produtos_estoque ON produtos (estoque);*

Índice composto por itens vendidos: *CREATE INDEX idx_itens_vendidos_produto_data ON itens_vendidos (ID_Produtos, data_venda);*

Garantir os índices das tabelas fks: *CREATE INDEX idx_produtos_id_categoria ON produtos (ID_Categoria); CREATE INDEX idx_produtos_id_estoque ON produtos (ID_Estoque);*

13.2 Configuração de Backup e Log

O sistema 4 TAGH usa uma estratégia de backup estruturada em três níveis: incremental, diário e semanal, garantindo diferentes pontos de recuperação e proteção dos dados.

- **Frequência de Backup:**

Backup Incremental (a cada 4 horas): Este backup captura apenas as mudanças realizadas desde o último backup, sendo executado quatro vezes ao dia durante o horário comercial (aproximadamente às 8h, 12h e 16h, além do backup completo noturno). A implementação utiliza tanto o MySQL Enterprise Backup da Oracle quanto o backup nativo do MySQL, com os dados sendo armazenados no servidor principal e imediatamente replicados para o datacenter secundário. A escolha do modelo incremental foi feita pensando em minimizar a perda de dados em caso de incidentes, limitando o impacto a no máximo 4 horas de trabalho. Estes backups são retidos por 7 dias, permitindo recuperação granular dentro da última semana operacional.

Backup Diário Completo: Realizado ao final de cada dia útil, fora do horário comercial (geralmente após as 20h), este backup captura o estado completo do banco de dados. A decisão por backups completos diários, ao invés de apenas semanais, foi feita pensando nos seguintes benefícios: oferece maior controle sobre as atividades dos funcionários, permite monitoramento mais preciso das alterações no sistema e proporciona múltiplos pontos de restauração. Por exemplo, se um problema é identificado na quarta-feira, a recuperação pode partir do backup de terça-feira, minimizando a perda de dados em comparação com um backup semanal que remontaria à sexta-feira anterior. Esses backups são mantidos por 60 dias (2 meses), oferecendo um histórico sólido para recuperação de médio prazo. A implementação dupla (MySQL Enterprise Backup e backup nativo) garante redundância e flexibilidade na escolha do método de restauração mais adequado a cada situação.

Backup Semanal Completo: Executado toda sexta-feira após o expediente, este backup funciona como uma camada adicional de segurança e serve como ponto de verificação semanal. Sua principal função é atuar como salvaguarda caso alguma inconsistência ou corrupção de dados passe despercebida nos backups diários, além de servir como baseline para análises de longo prazo. Com retenção de 12 meses (1 ano), estes backups atendem também requisitos de auditoria e conformidade, permitindo investigações retroativas e análises históricas do sistema. A implementação segue o mesmo padrão dual (MySQL Enterprise Backup e nativo), garantindo consistência metodológica em toda a estratégia de backup.

- **Tipo de Backup:**

Backup incremental: copia apenas os dados que foram alterados desde o último backup.

Backup Completo: copia todos os dados do sistema.

- **Armazenamento e Proteção Geográfica:**

Todos os backups são automaticamente replicados para um datacenter secundário da empresa, protegendo contra desastres físicos no local principal, como incêndios, inundações ou falhas de hardware. A sincronização acontece em tempo real para os backups incrementais e logo após a finalização dos backups completos, garantindo que sempre exista uma cópia segura dos dados em localização separada.

- **Política de Logs:**

O sistema mantém ativados os binary logs (binlog) do MySQL para permitir recuperação precisa de dados. Configurados no formato ROW, estes logs registram todas as operações realizadas no banco (INSERT, UPDATE, DELETE) e são mantidos por 7 dias, sendo sincronizados a cada 6 horas para o armazenamento secundário. A importância dos binlogs está na capacidade de fazer Point-in-Time Recovery: mesmo que um problema aconteça entre dois backups incrementais, é possível restaurar o banco até o momento exato anterior ao incidente. Por exemplo, se às 14h30 ocorre uma exclusão acidental de dados, pode-se restaurar o backup incremental das 12h e aplicar os binary logs até às 14h29, recuperando tudo até o minuto antes do erro, sem perder trabalho válido realizado naquele período.

- **Procedimentos de Recuperação:**

O sistema trabalha com três cenários principais. Para recuperação completa após corrupção total do banco, usa-se o último backup diário completo, levando cerca de 2 a 4 horas. Para recuperação pontual de erros específicos, combina-se o backup incremental mais recente com os binary logs, permitindo PITR em 1 a 2 horas. Em caso de desastre no datacenter principal, aciona-se o backup semanal do datacenter secundário, com tempo de recuperação entre 6 e 12 horas. A equipe faz testes mensais de restauração para validar que os backups estão funcionando corretamente e manter o time treinado nos procedimentos.

- **Monitoramento e Validação:**

Um sistema automatizado monitora todos os processos de backup, verificando se foram concluídos com sucesso, validando a integridade através de checksums e emitindo alertas imediatos caso algo falhe. Um dashboard mostra em tempo real o status de todos os backups, incluindo horário da última execução, tamanho dos arquivos e tempo de processamento, permitindo identificar problemas rapidamente. Esta estratégia define um RPO (Recovery Point Objective) de 4 horas – o máximo de dados que pode ser perdido em um incidente – e um RTO (Recovery Time Objective) de 4 horas para recuperação completa do sistema, garantindo que o impacto operacional de qualquer problema seja minimizado.

Declaração de Autoria

Declaramos que este trabalho foi integralmente desenvolvido por nossa equipe.

Integrante 1: Allan Takeshi Morimoto



Assinatura:

Integrante 2: Heitor Alves Dos Santos



Assinatura:

Integrante 3: Gustavo Vinicius Claman de Andrade



Assinatura:

Integrante 4: João Rafael K. Andrade



Assinatura:

Integrante 5: Tiago Muraro de Paula Gomes



Assinatura: